



UNIVERSIDADE ESTADUAL DE CAMPINAS
Faculdade de Engenharia Química

Luiz Henrique Silva Albino

Thiago Ferrazin Fernandes

**Detecção de falhas em poços de petróleo: algoritmos para
aprimorar a análise em sistema de elevação.**

Campinas
2023

Luiz Henrique Silva Albino
Thiago Ferrazin Fernandes

Deteccção de falhas em poços de petróleo: algoritmos para aprimorar a análise em sistema de elevação.

Trabalho de Conclusão de Curso apresentado à Faculdade de Engenharia Química da Universidade Estadual de Campinas como parte dos requisitos para a obtenção do título de Bacharel em Engenharia Química.

Supervisor: Prof. Dr. Flávio Vasconcelos da Silva

Este trabalho corresponde à versão final do Trabalho de Conclusão de Curso defendido por Luiz Henrique Silva Albino e por Thiago Ferrazin Fernandes e supervisionado pelo Prof. Dr. Flávio Vasconcelos da Silva.

Campinas
2023

DECLARAÇÃO DE AUTORIA

Nós, Luiz Henrique Silva Albino e Thiago Ferrazin Fernandes, declaramos que esse trabalho é de autoria própria e que as partes que não o são foram referenciadas ao longo de todo o trabalho.

Campinas, 22 de novembro de 2023.

Agradecimentos

Na trajetória de ambos os autores, há muitas pessoas que gostaríamos de agradecer, porém, vamos ressaltar àquelas que contribuíram durante o processo de construção deste projeto.

Ao Prof. Dr. Flávio Vasconcelos, que além de atuar como supervisor do projeto, merece reconhecimento por sua flexibilidade no alinhamento de agendas e pelas valiosas sugestões de aprimoramento e direcionamento para o projeto. Profissionais desse calibre têm um impacto significativo na transformação das sociedades.

Ao Prof. Dr. Sávio Vianna, que nos auxiliou durante o processo de formulação do projeto, e foi responsável por nos trazer uma visão cética sobre o trabalho.

À Dr. Ana Cláudia Souza, que ao nos apresentar o seu trabalho de doutorado, nos introduziu o projeto do *3W Dataset* e trouxe diretrizes inestimáveis para a conciliação da proposta.

Ao Dr. Ricardo Vargas, que fez um trabalho excepcional ao estruturar a base de dados de dados industriais de uma planta de extração de petróleo. A sua tese de doutorado e o repositório no *Github* foram imprescindíveis para a execução do trabalho.

Ao youtuber e fundador da *Tech With Tim Inc.*, Tim Ruscica, que foi a referência dos autores para o desenvolvimento da aplicação web.

Realmente, muito obrigado!

When do you think a person dies? When a bullet from a pistol pierces his heart? No. When he's attacked by an incurable disease? No. When he eats a soup of deadly poisonous mushrooms? No. A man dies when people forget him!.

(Dr. Hiriluk)

Resumo

A Indústria 4.0 marcou a integração dos Controladores Lógicos Programáveis (CLP) e sistemas digitais, diminuindo a intervenção humana e agilizando a tomada de decisão. No entanto, a crescente complexidade dos sistemas de instrumentação, a demanda por qualificação profissional e a maturidade tecnológica se tornaram desafios. Especialmente na fase de pré-processamento de petróleo, a precisão dos dados coletados por sensores se tornou crítica. Gerenciar eventos anormais em poços de petróleo é crucial para minimizar custos de manutenção e evitar perdas de produção, especialmente em ambientes marítimos. Este trabalho fez uma análise de detecção de falhas a partir do *3W Dataset*, referente ao processo de elevação da Unidade de Operação da Petrobras (UO-ES). Os modelos de aprendizado de máquina aplicados apresentaram desempenho satisfatório na classificação de falhas, com exceção da Regressão Logística, que teve dificuldades em generalizar eventos de falha. Entre os algoritmos avaliados, os K Vizinhos Mais Próximos, a Floresta Aleatória e o Perceptron Multicamadas se destacaram. A média das acurácias e pontuação F1 para os K Vizinhos Mais Próximos foi de 98% e 98%, para a Floresta Aleatória de 93% e 93% e para o Perceptron Multicamadas de 89% e 87%. No que tange ao tempo de execução, o algoritmo de Máquinas de Vetores de Suporte (SVM) apresentou os maiores tempos, inviabilizando sua aplicação em cenários que requerem respostas rápidas. Em contrapartida, os K Vizinhos Mais Próximos tiveram o menor tempo médio de processamento, com um valor de 12 segundos. Outra contribuição desse trabalho foi o desenvolvimento da aplicação física dos modelos testados, foi desenvolvida a plataforma *Detecção de Falhas em Poços e Sistemas de Elevação (DFPSE)*. A plataforma foi desenvolvida utilizando o *framework Flask* da linguagem *Python*, de modo a demonstrar que é possível democratizar o uso de tais tecnologias para Engenheiros de Processos.

Palavras-chave: Controle de Processos, Detecção de falhas, Monitoramento de poços de petróleo, Aprendizado de Máquina, 3W Dataset, Petrobras.

Abstract

The Industry 4.0 marked the integration of Programmable Logic Controllers (PLCs) and digital systems, reducing human intervention and expediting decision-making. However, the increasing complexity of instrumentation systems, the demand for professional qualification, and technological maturity have become challenges. Especially in the oil preprocessing phase, the accuracy of data collected by sensors has become critical. Managing abnormal events in oil wells is crucial to minimize maintenance costs and prevent production losses, especially in maritime environments. This work conducted a fault detection analysis using the *3W Dataset*, related to the lifting process of Petrobras Operational Unit (UO-ES). The applied machine learning models demonstrated satisfactory performance in fault classification, except for Logistic Regression, which struggled to generalize fault events. Among the evaluated algorithms, K-Nearest Neighbors, Random Forest, and Multilayer Perceptron stood out. The average accuracies and F1 scores for K-Nearest Neighbors were 98% and 98%, for Random Forest 93% and 93%, and for Multilayer Perceptron 89% and 87%. Regarding execution time, the Support Vector Machine (SVM) algorithm had the longest processing times, rendering it impractical for scenarios requiring quick responses. Conversely, K-Nearest Neighbors had the shortest average processing time, at 12 seconds. Another contribution of this work was the development of a physical application for the tested models, resulting in the *Fault Detection in Wells and Lifting Systems (DFPSE)* platform. The platform was built using the Flask framework in the Python language, demonstrating the feasibility of making such technologies accessible to Process Engineers.

Keywords: Process Control, Fault Detection, Oil Well Monitoring, Machine Learning, 3W Dataset, Petrobras.

Lista de Figuras

1.1	Ilustração das quatro etapas do transporte de petróleo (ANDREOLLI, 2016). . .	16
1.2	Exemplo dos métodos de elevação citados Fonte: Adaptado de (NASCIMENTO, 2005)	18
2.1	Classificação de métodos baseados em dados históricos (VENKATASUBRAMANIAN; RENGASWAMY; KAVURI et al., 2003)	21
2.2	Exemplo de matriz de confusão. Fonte (KULKARNI; CHONG; BATARSEH, 2020)	25
2.3	Exemplo de curva ROC com múltiplos classificadores. Fonte Wikipedia	26
2.4	Exemplo para método de classificação com algoritmo dos K Vizinhos. Fonte: (ATALLAH; BADAWY; EL-SAYED, 2019)	27
2.5	Exemplo de Árvore de Decisão	28
2.6	Unidade linear com Threshold (GÉRON, 2022).	30
2.7	Um MLP moderno para classificação (incluindo ReLU e Softmax) (GÉRON, 2022).	31
3.1	Histograma das variáveis T-TPT e P-TPT Fonte: Autores.	36
4.1	Matriz de Confusão para cada tipo de Evento utilizando Regressão Logística Múltipla.	40
4.2	Matriz de Confusão para cada tipo de Evento utilizando K Vizinhos Mais Próximos.	42
4.3	Matriz de Confusão para cada tipo de Evento utilizando Floresta Aleatória. . .	44
4.4	Matriz de Confusão para cada tipo de Evento utilizando Máquinas de vetores de suporte.	46
4.5	Matriz de Confusão para cada tipo de Evento utilizando Máquinas de vetores de suporte.	48
4.6	Visão da página inicial da Aplicação DFPSE.	49
4.7	Visão da página de escolha do <i>dataset</i> da Aplicação DFPSE.	49
4.8	Visão da página de análise exploratória do <i>dataset</i> da Aplicação DFPSE. . . .	50
4.9	Visão dos resultados dos modelos para o Evento 2.	50

Lista de Tabelas

4.1	Métricas de desempenho do modelo de Regressão Logística Múltipla para diferentes eventos.	39
4.2	Métricas de desempenho do modelo de K Vizinhos Mais Próximos para diferentes eventos.	41
4.3	Métricas de desempenho do modelo de Floresta Aleatória para diferentes eventos.	43
4.4	Métricas de desempenho do modelo de Máquinas de vetores de suporte para diferentes eventos.	45
4.5	Métricas de desempenho do modelo de Perceptron Multicamadas para diferentes eventos.	47

Lista de Símbolos

<i>AUC</i>	<i>Area Under Curve</i>
<i>CKGL</i>	<i>Choke de Gas Lift</i>
<i>CKP</i>	<i>Choke de Produção</i>
<i>CLP</i>	Controladores Lógicos Programáveis
<i>CPS</i>	<i>Cyber Phisical System</i>
<i>DSV</i>	<i>Downhole Safety Valve</i>
<i>DHSV</i>	<i>Down Hole Safety Valve</i>
<i>F1</i>	Pontuação F1 ou <i>F1-Score</i>
<i>FNN</i>	Rede Neural <i>feedforward</i>
<i>FN</i>	<i>False Negative</i>
<i>IoT</i>	<i>Internet of Things</i>
<i>KNN</i>	K Vizinhos Mais Próximos
<i>LTU</i>	Unidade Linear com Threshold
<i>MLP</i>	<i>Multi Layer Perceptron</i>
<i>PCA</i>	<i>Principal Component Analysis</i>
<i>PDG</i>	<i>Permanent Downhole Gauge</i>
<i>P</i>	Precisão
<i>P-MON-CKP</i>	Pressão do fluido montante à válvula CKP
<i>P-PDG</i>	Pressão do fluido no PDG
<i>Pr</i>	Probabilidade de um evento acontecer
<i>P-TPT</i>	Pressão do fluido no TPT
<i>QGL</i>	Vazão do gás usado para elevação
<i>RBF</i>	Função de Base Radial Gaussiana
<i>RNA</i>	Redes Neurais Artificiais
<i>ROC</i>	<i>Receiver Operation Curve</i>
<i>SVM</i>	Máquinas de Vetores de Suporte
<i>Sn</i>	Sensibilidade ou <i>Recall</i>
<i>T-JUS-CKP</i>	Temperatura do fluido jusante à válvula CKP
<i>T-TPT</i>	Temperatura do fluido no TPT
<i>TN</i>	<i>True Negative</i>
<i>TP</i>	<i>True Positive</i>

Sumário

1	Introdução	13
1.1	Falhas e eventos anômalos:	14
1.2	O processo de elevação e escoamento de petróleo	15
1.3	Objetivos	19
2	Revisão bibliográfica e Fundamentação Teórica	20
2.1	Fundamentação Teórica	20
2.1.1	Funcionamento de algoritmos de aprendizado de máquina	20
2.2	Algoritmos de Aprendizado de Máquina	20
2.2.1	Diagnóstico e predição de falhas na engenharia de processos	20
2.2.2	Regressão Logística e Classificadores	23
2.2.3	Regressão Logística Múltipla	26
2.2.4	K Vizinho Mais Próximos	27
2.2.5	Árvores de Decisão e Florestas Aleatórias	28
2.2.6	Máquinas de vetores de suporte (SVM)	28
2.2.7	Redes Neurais Artificiais - Perceptrons Multicamadas (MLP)	29
2.3	Revisão Bibliográfica	32
3	Metodologia	34
3.1	3W Dataset	34
3.2	Modelos de Classificação	36
3.3	Aplicação	37
4	Resultados e Discussão	38
4.1	Regressão Logística Múltipla	38
4.2	K Vizinhos Mais Próximos	41
4.3	Floresta Aleatória	42
4.4	Máquinas de vetores de suporte (SVM)	45
4.5	Perceptron Multicamadas - Redes Neurais Artificiais	47
4.6	Aplicação DFPSE - Detecção de Falhas em Poços e Sistemas de Elevação	49
5	Conclusão	52
	Referências bibliográficas	54
A	Primeiro Apêndice	57
A.1	Modelos de Aprendizado de Máquina e Indicadores de Desempenho	57
A.2	Rotinas para ler a base de dados e concatenar Arquivos do mesmo evento:	61

B	Segundo Apêndice	64
B.1	Arquivo principal para aplicacao DFPSE	64
B.2	Funcoes utilizadas dentro da Aplicacao DFPSE	70

Capítulo 1

Introdução

A terceira revolução industrial teve início após o final da segunda guerra mundial. Com ela, a indústria observou uma transformação com o emprego dos controladores lógicos programáveis (CLP) e a diminuição da necessidade de ação humana na manutenção de um processo em condições ótimas [Santos, Manhães e Lima \(2018\)](#). Com o avanço da integração com internet, a humanidade passa a experimentar o surgimento da Indústria 4.0.

O advento e evolução das técnicas trazidas junto com a mais recente revolução industrial levantam a demanda e interesse geral por segurança operacional e manutenção da produtividade. A indústria passa a experimentar uma conexão maior entre sistemas físicos e digitais. Ferramentas como Sistemas Ciber-Físicos (CPS - *Cyber Physical System*) acoplam sensores, controladores, sistemas de armazenamento de dados e comunicação através de *IoT (Internet of Things)* possibilitando a atuação e troca de informações dentro de setores da planta de maneira mais segura e automatizada, mitigando a necessidade de atuação manual e auxiliando na tomada de decisão de maneira rápida e assertiva. ([SANTOS; MANHÃES; LIMA, 2018](#)).

Entretanto, conforme destacado por [Venkatasubramanian, Rengaswamy, Yin et al. \(2003\)](#) o avanço e possibilidades caminham junto com o aumento da complexidade requerida pelos sistemas de instrumentação, qualificação profissional e maturidade tecnológica. Essa complexidade é necessária para realização de tarefas, antes impraticáveis, como o gerenciamento, detecção, prevenção e prognóstico de eventos anormais.

Um dos pontos de extrema relevância na indústria do petróleo e gás se verifica durante a fase inicial de pré-processamento do petróleo, pois os dados coletados pelos sensores associados ao sistema CPS estão suscetíveis a imprecisões inerentes aos próprios dispositivos ou aos sistemas com os quais estão interconectados, o que pode prejudicar avaliações que dependem dessas

informações. Portanto, é imperativo desenvolver abordagens de mitigação de erros e aprimorar a confiabilidade desses dados, garantindo análises mais sólidas e confiáveis (MOMM, s.d.).

Uma gestão eficaz de eventos anormais em poços de petróleo desempenha um papel essencial na minimização dos custos associados à manutenção e na prevenção de perdas de produção, bem como na redução de riscos de acidentes ambientais e de segurança. Essa responsabilidade ganha ainda mais relevância em poços marítimos, uma vez que suas operações, riscos inerentes e despesas costumam ser substancialmente ampliados. Portanto, a implementação de estratégias sólidas de gerenciamento de eventos anormais é crucial para garantir a integridade, eficiência e segurança dessas instalações petrolíferas (VARGAS, 2019).

Ainda, após a distribuição comercial de sistemas em nuvem (*cloud computing*) na primeira década do século XX, a adoção crescente de ferramentas de computação distribuída, sistemas de armazenamento em nuvem *Data Lake*, geração de grandes volumes de dados *Big Data*, permitiu a aplicação de ferramentas robustas em diversos contextos econômicos.

Exemplos, como a aplicação de computação distribuída com algoritmos de otimização na extração de cafeína com dióxido de carbono supercrítico desenvolvido por Galeazzi et al. (2021), aplicação de Redes Neurais e *Deep Learning* para o monitoramento e detecção de falhas em turbinas de geração eólica Helbing e Ritter (2018) e muitos outros casos, ressaltam a tendência de modernização e de fusão do mundo da computação e de processos. (TRIGUEROS-PRECIADO; PÉREZ-GONZÁLEZ; SOLANA-GONZÁLEZ, 2013)

1.1 Falhas e eventos anômalos:

A definição de falha, de acordo com Aldrich e Auret (2013), é um comportamento anormal de um processo que o retira de um estado aceitável de suas condições de operações normais. Ainda, cada falha pode ter associado consigo um evento iniciador, ou causa raiz, como um mal funcionamento de um atuador, falha de calibração de um sensor, falha de controlador, entre outras.

A detecção precoce de falhas em regiões em que o processo ainda se encontra em regiões estáveis é crucial para impedir desdobramentos críticos, paradas de processos e eventos descontrolados em linhas de produção. Dado que a indústria petroquímica perde, aproximadamente, 20 bilhões d/e dólares ao ano (VENKATASUBRAMANIAN; RENGASWAMY; YIN et al., 2003), a

detecção e predição de falhas despertam considerado interesse tanto de pesquisa acadêmica quanto de aplicação para mitigar perdas.

Através do desenvolvimento e aprimoramento dos sistemas de controle de processos, armazenamento de dados e poder computacional, novas abordagens foram desenvolvidas para a análise e gerenciamento de eventos anômalos no escopo da engenharia de processos.

1.2 O processo de elevação e escoamento de petróleo

Desde a descoberta do potencial energético do petróleo como principal componente da matriz energética moderna, os profissionais da indústria buscam maneiras de se aprofundar e destrinchar os detalhes acerca do processo de extração do mesmo nos mais diversos ambientes, muitas vezes em condições extremas de temperatura e pressão.

Em parte dos poços de petróleo, a substância se encontra em determinado estado que a elevação do mesmo se faz sem a necessidade de realização de trabalho mecânico, tais poços são denominados de poços surgentes (*flowing wells*). (ANDREOLLI, 2016)

Porém, a pressão interna do poço decai com a retirada de material do mesmo. Assim, fazendo uma analogia com a equação de Bernoulli, pode-se inferir que a vazão do poço também reduzirá com o tempo. Dessa forma, se faz necessária a aplicação de trabalho mecânico externo ao sistema delimitado pelo poço. Diversas técnicas podem ser utilizadas, à depender do sistema, tamanho deste, potência requerida, razão gás-líquido, vazão de água e areia entre outros fatores.

A extração de petróleo em ambientes marítimos e terrestres é realizada por meio de estruturas chamadas poços de petróleo. Esses poços consistem em conjuntos de sensores e sistemas mecânicos, elétricos e hidráulicos. Em poços marítimos, instalados no leito marinho ou na superfície, e em poços terrestres, instalados na superfície da terra, esses equipamentos desempenham a função de interligar um reservatório de petróleo a uma instalação industrial.

A instalação industrial em poços *offshore* é uma unidade estacionária marítima de produção, enquanto em poços *onshore* é uma unidade de produção terrestre. Essas unidades realizam um pré-processamento do petróleo, separando-o em óleo, gás e água, de acordo com as especificações da Agência Nacional do Petróleo, Gás Natural e Biocombustíveis (ANP). A água é reutilizada ou descartada, e o óleo e o gás são transportados para refinarias ou unidades de tratamento de gás (ANDREOLLI, 2016).

Existem dois tipos principais de poços de petróleo: injetor e produtor. Os poços injetores são destinados à injeção de água, gás ou produtos químicos em um reservatório, com o objetivo de aumentar a produção dos poços. Já os poços produtores são responsáveis por transportar o petróleo do reservatório até a instalação industrial. A área de engenharia conhecida como Elevação e Escoamento de Petróleo trata do transporte do petróleo desde o reservatório até refinarias ou unidades de tratamento de gás (ANDREOLLI, 2016).

Figura 1.1: Ilustração das quatro etapas do transporte de petróleo (ANDREOLLI, 2016).

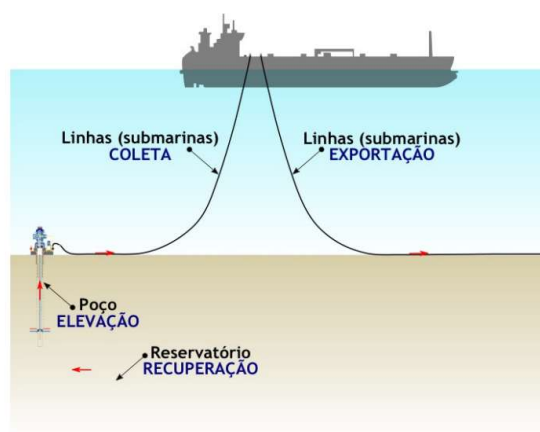


Foto adaptada pelo autor.

O transporte de petróleo é dividido em quatro etapas: recuperação, elevação, coleta e exportação. Cada etapa apresenta características específicas e diferentes fenômenos podem ocorrer em cada uma delas. (VARGAS, 2019)

- **Recuperação:** ocorre no meio poroso da rocha reservatório, influenciada pelas características da rocha e do fluido.
- **Elevação:** ocorre na coluna de produção dos poços produtores, com a gravidade terrestre sendo o principal fator que influencia o fluxo.
- **Coleta:** Etapa mais comum do escoamento em poços marítimos, ocorre na porção do mar entre a plataforma e o poço. Normalmente dividido em duas etapas, horizontal (próximo ao leito do mar) chamada de *flowline* e uma vertical (do leito à plataforma) chamada de *riser*
- **Exportação:** Momento final do transporte, ocorre normalmente entre uma unidade produtora e uma unidade de processamento, refinaria ou outro navio coletor de óleo e gás

Os sistemas de elevação e escoamento de petróleo mais comuns na atualidade, levando-se em consideração todo o globo, são brevemente descritos a seguir, um diagrama explicativo de cada método pode ser observado na Figura 1.2

- **Bombeamento Mecânico:** Logo após o surgimento da exploração comercial do petróleo, o método de bombeamento ou *Sucker-Rod Pumping* foi desenvolvido. Atualmente, de acordo com Barros Filho (2002) representa 8% da produção diária de petróleo, além de ser o mais presente no mundo. Sendo muito útil em poços de baixas vazões e pouca profundidade, requerendo baixo custo inicial de investimento e manutenção.
- **Gas lift ou Injeção de Gás:** Predominante em poços injetores, esse método consiste na separação da própria fase gasosa presente no poço de sua fase líquida, sua recompressão e injeção novamente dentro do poço. Assim, a pressão interna do poço aumenta e auxilia na compensação de perda de carga do escoamento do fluido através das tubulações.
- **Bombeio por Cavidades Progressivas:** Através de um sistema similar ao funcionamento de bombas de deslocamento positivo, o Bombeio por Cavidades Progressivas funciona através da rotação de hastes internas que geram seções de cavidades com mesma seção que, ao completar uma rotação, deslocam uma porção do fluido.
- **Plunger Lift:** Método associado à acumulação de uma coluna de fluidos é acumulada acima de um pistão e, com a abertura de uma válvula na superfície, há a expansão do gás presente no interior, empurrando o pistão para cima e, por consequência elevando os fluidos restantes

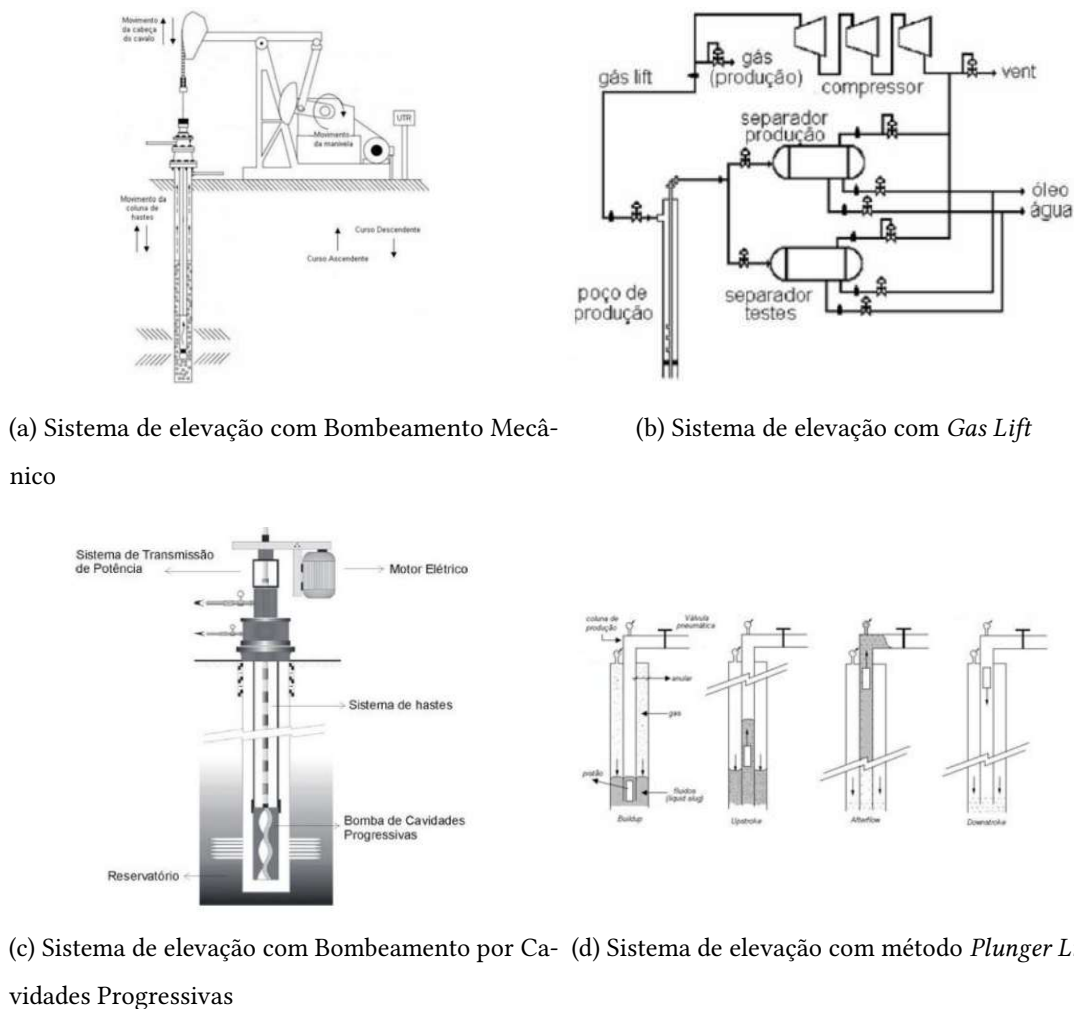


Figura 1.2: Exemplo dos métodos de elevação citados Fonte: Adaptado de (NASCIMENTO, 2005)

Cada método de elevação tem suas características e aplicabilidade específicas, sendo escolhido de acordo com as condições do reservatório e os requisitos de produção do poço (VARGAS, 2019).

Ainda, a questão de segurança em plataformas *offshore* é latente ainda nos dias atuais devido à sua magnitude, riscos ambientais e de vida aos operadores embarcados. O acidente na plataforma em Macondo (Estados Unidos) em 2010 resumem o impacto que o mal funcionamento e falhas podem gerar em sistemas de elevação. No acidente ocorreu o afundamento da sonda *Deep Water Horizon*, morte dos onze trabalhadores e danos ambientais em um dos piores acidentes dos Estados Unidos. (SUTTON, 2013). Em questões de manutenção, uma sonda adaptada para trabalhar em plataformas pode custar até US\$ 500 mil dólares, levantando a importância de conseguir monitorar, prever e agir proativamente na mitigação desses problemas. (ANDREOLLI, 2016)

1.3 Objetivos

O presente trabalho tem como objetivos gerais a compreensão das variáveis mais importantes na detecção e gerenciamento de eventos anormais através de comparação entre modelos estatísticos e de aprendizado de máquina para análise do *dataset* 3W da Petrobras.

O estudo busca comparar alguns modelos, como: Regressão Logística, *Random Forest*, K Vizinhos Mais Próximos (KNN), Máquinas de Vetores de Suporte (SVM), Redes Neurais (Perceptron Multicamadas).

Além disso, o estudo conta com a construção de um protótipo de aplicação web, que busca demonstrar como algoritmos de detecção de falhas, quando otimizados, podem ser utilizados em uma rotina de análise de dados de um engenheiro de processos ou de segurança, por exemplo.

Capítulo 2

Revisão bibliográfica e Fundamentação Teórica

2.1 Fundamentação Teórica

2.1.1 Funcionamento de algoritmos de aprendizado de máquina

Segundo [Géron \(2022\)](#), Aprendizado de Máquina é a ciência (e a arte) da programação de computadores para que eles aprendam com dados. Nos algoritmos de Aprendizado de Máquina, o aprendizado é adquirido e armazenado na forma de hipóteses, funções ou modelos para resolver problemas específicos. Essa aprendizagem é obtida a partir de exemplos fornecidos, chamados de instâncias, e pode ser aplicada a diversas situações diferentes. ([VARGAS, 2019](#)).

O processo de aprendizado de máquina envolve duas fases: a fase de treinamento, na qual a hipótese, função ou modelo é induzido, e a fase de utilização, quando o aprendizado é aplicado para resolver problemas. Algoritmos de aprendizado podem ser organizados de várias maneiras, geralmente seguindo paradigmas específicos. Essa organização influencia a hierarquia e a quantidade de categorias e níveis de profundidade. ([FACELI et al., 2021](#)).

2.2 Algoritmos de Aprendizado de Máquina

2.2.1 Diagnóstico e predição de falhas na engenharia de processos

[Venkatasubramanian, Rengaswamy, Kavuri et al. \(2003\)](#) demonstra que, junto com esse desenvolvimento, procedimentos de *feature extraction* (extração de componentes) se fez possível

com a utilização de dados históricos de operação, em comparação com a complexidade da abordagem fenomenológica. Ainda nesse âmbito, duas principais vertentes para resolução de problemas de engenharia de processos com base em dados nascem: Qualitativos e Quantitativos 2.1.

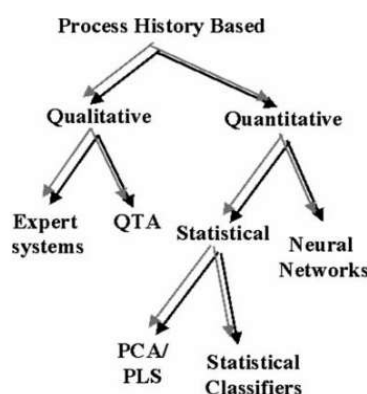


Figura 2.1: Classificação de métodos baseados em dados históricos (VENKATASUBRAMANIAN; RENGASWAMY; KAVURI et al., 2003)

Os sistemas especialistas se baseiam na delimitação de regras e são aplicadas em amplos contextos da engenharia de processos. Os principais conceitos deste são: Aquisição de conhecimento, escolha de representação do conhecimento, codificação do conhecimento em uma base de conhecimento, desenvolvimento de procedimentos de inferência para raciocínio diagnóstico e desenvolvimento de interfaces de entrada/saída. A principal vantagem dos sistemas especialistas são o desenvolvimento claro das inferências utilizadas, a fácil codificação e a possibilidade de fornecer explicações claras para o contexto. Entretanto, por estarem dentro de um domínio muito específico, suas soluções podem ser de difícil generalização ou transposição para outros cenários. As primeiras propostas de sistemas especialistas aplicados em problemas de processos químicos remonta da década de 80 no artigo escrito por Kumamoto et al. (1984) onde um sistema de regras, codificado através árvores de decisão *if-else* (se-senão), é proposto para o controle de um sistema de refrigeração através de 22 regras sobre condições operacionais atreladas à 16 causas prováveis de falha do sistema.

A *Qualitative Trend Analysis (QTA)*, ou Análise de Tendência Qualitativa, permite a identificação de uma modificação de estado normal de operação de um processo com base na variação temporal de uma propriedade (sua derivada) e seu valor atual. (CHEUNG; STEPHANOPOULOS, 1990)

Apesar de amplamente estudados, há uma limitação clara na aplicação de um sistema especialista. A definição de regras claras e que cubram todo o escopo de operação podem ser complexos e se tornarem uma tarefa árdua, entretanto sua aplicação ainda pode ser vantajosa em alguns casos. Em contrapartida, os métodos quantitativos podem oferecer certas vantagens nesse escopo.

A natureza dos processos químicos se traduz, muitas vezes, em dinâmicas de alta ordem e não lineares. Analisar diversos fatores ao mesmo tempo pode prejudicar, além de tornar moroso, o processo de investigação de um evento anômalo. Para isso, a Análise de Componente Principal (*Principal Component Analysis*) se torna uma poderosa ferramenta.

Em termos gerais, a PCA é utilizada para decompor um problema multivariável em componentes ortogonais da matriz de covariância das variáveis do problema analisado. Assim, é possível determinar, através de manipulações matemáticas, quais fatores identificam as principais macro tendências do processo e reduzir a dimensão do problema em suas componentes mais importantes.

A aplicação da PCA se estende por diversos problemas e campos diversos com o intuito de suportar análises multivariadas. Por não ser utilizada na resolução do problema deste trabalho, a PCA não será abordada com detalhes tão aprofundados, limitando-se a discorrer sobre sua importância histórica. Entretanto, a fundamentação e aplicação da teoria da Análise de Componentes Principais é descrita no artigo de [Bro e Smilde \(2014\)](#).

Os classificadores estatísticos (*Statistical Classifiers*) partem do conceito inicial de um modelo matemático que define com qual espaço o evento i está relacionado. O Classificador de Bayes é considerado um dos primeiros estimadores utilizados. De acordo com ([VENKATASUBRAMANIAN; RENGASWAMY; KAVURI et al., 2003](#)), um estimador de Bayes onde as funções de densidade seguem a distribuição Gaussiana é dado por:

$$d_1 == (y - m_1)^T \Sigma_1^{-1} (y - m_1) \quad (2.1)$$

$$d_2 == (y - m_2)^T \Sigma_2^{-1} (y - m_2) \quad (2.2)$$

$$\delta = \ln \left(\frac{|\Sigma_2|}{|\Sigma_1|} \right) \quad (2.3)$$

Onde d_1 e d_2 representam as distâncias de um ponto em um espaço com base em suas coordenadas, sua diferença, h , representa a relação entre a distância 1 e 2. O valor de δ

representa o *threshold* (limite de corte) para o classificador. Sendo assim, se $h < d_1$ o ponto pertence à classe 1, caso contrário ele pertence à classe 2.

Novos modelos estatísticos com base quantitativa e na extração de histórico de dados foram desenvolvidos no decorrer do século XX, junto com o avanço do poder computacional disponível. Na seção de Metodologia, os algoritmos de classificadores estatísticos como Regressão Logística, KNN (k Vizinhos mais Próximos), Floresta Aleatória e Máquina de Vetores de Suporte serão analisados. Além disso, uma aplicação de Rede Neural (*Neural Network*) com a arquitetura de *Multi Layer Perceptron* também será abordada.

2.2.2 Regressão Logística e Classificadores

O modelo de regressão logística foi proposto por Cox em 1958 no artigo *The regression analysis of binary systems*. Neste, o autor discorre sobre uma série de análises, ou testes, realizados em cima de um conjunto de amostras onde o resultado possível dela é um intervalo $y \in [0,1]$ discreto. Os valores podem possuir representações físicas como "falha" ou "normal" para o contexto de processos, "positivo" ou "negativo" para o caso infectológico, "sucesso" ou "falha" para eventos, "ligado" ou "desligado" para sinais. Um modelo comumente atrelado ao tema de regressão é o Linear, que segue o formato:

$$Pr\{Y = 1|x, \beta\} = \beta_0 + \beta_1 \cdot x_1 \quad (2.4)$$

Onde Pr representa a probabilidade de $Y = 1$ para um dado conjunto de variáveis x e parâmetros β . Entretanto, o modelo acima necessita de uma correção uma vez que o lado esquerdo da equação está delimitado pelo intervalo $[0,1]$ enquanto o lado direito oscila, para os números reais, entre $[-\infty, \infty]$.

A formulação proposta por Cox (1958) se baseia na consideração da relação do *log* da chance de determinação da probabilidade da variável dependente ser categorizada, por exemplo, na categoria 1, $Y_i = 1$, dado um vetor de variáveis independentes x_i e parâmetros β_i . Considerando apenas uma única variável independente e parâmetro temos a transformação do termo de *odds*, ou chance, para a adequação matemática dos limites da equação acima.

$$\pi_i = \frac{Pr\{Y = 1|x, \beta\}}{1 - Pr\{Y = 1|x, \beta\}} \quad (2.5)$$

Por fim, para adequar o termo da Equação 2.5 na Equação 2.4 é necessária a transformação:

$$\log \frac{\Pr\{Y = 1|x, \beta\}}{1 - \Pr\{Y = 1|x, \beta\}} = \log \frac{\pi_i}{1 - \pi_i} = \beta_0 + \beta_1 \cdot x_1 \quad (2.6)$$

Assim, o modelo proposto por Cox demonstra uma relação entre o logaritmo da razão de chances (*odds*) e os parâmetros de maneira linear. Aplicando transformações em ambos os lados e aplicação de notação matricial para β e x , o modelo resulta em:

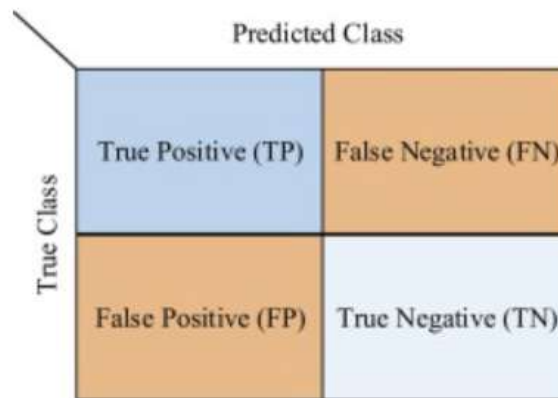
$$\pi_i(Y = 1) = \frac{e^{\beta^T \cdot x}}{1 + e^{\beta^T \cdot x}} \quad (2.7)$$

Com isso, é possível estimar os parâmetros para que o modelo consiga se adequar de melhor maneira ao dado do problema. Para isso, é possível utilizar uma porção de dados de treino e um função de otimização para que os β_i sejam definidos.

Porém, é interessante notar que o modelo proposto por Cox ainda não responde a dúvida inicial da classificação quando um novo dado é colocado. O retorno representa apenas uma probabilidade da observação i estar atrelada a um estado. Assim, ainda é necessária a consideração do ponto de corte (*threshold*) a ser implementada na classificação.

O ponto de corte significa o limite em que uma probabilidade, resultante de um dado não observado previamente pelo modelo, entre as duas categorias do modelo de regressão logística. Abstraindo com um exemplo mais cotidiano, pode-se imaginar que 0 significa um estado doente e 1 significa um estado saudável, ao realizar um teste sanguíneo modelado pela regressão logística, o resultado poderia ser algo como "Probabilidade de estar doente: 40%". Em alguns cenários, será interessante observar as probabilidades individuais dos eventos mas, para efeitos de Classificação é necessário termos uma resposta definitiva da categoria 0 ou 1.

Para definir o ponto de corte é utilizado métricas atreladas à matriz de confusão. A matriz tem como objetivo representar graficamente como o modelo se comporta com as classificações realizadas dado um ponto de corte α . A Figura 2.2 representa um exemplo para uma regressão binária, as classes preditas pelo modelo estão colocadas nas colunas enquanto as classes Verdadeiras representam as linhas.



		Predicted Class	
True Class	Actual Positive	True Positive (TP)	False Negative (FN)
	Actual Negative	False Positive (FP)	True Negative (TN)

Figura 2.2: Exemplo de matriz de confusão. Fonte ([KULKARNI; CHONG; BATARSEH, 2020](#))

Buscando entender acurácia do modelo, dois principais pontos surgem da análise da matriz de confusão: os Falsos Negativos e Falsos Positivos. Remetendo ao exemplo infectológico, Falso Negativo seria um paciente realmente doente porém com diagnóstico normal. Já o Falso Positivo representa um paciente Normal com diagnóstico doente. É claro o impacto que esse tipo de afirmação representa, em um contexto de processos químicos pode-se estar deixando passar falhas críticas em equipamentos, podendo causar acidentes ou fazendo intervenções para reparos em linhas de processo que operam normalmente, aumentando gastos, tempo de parada, manutenção, etc.

Dado os quatro quadrantes observados, é possível definir algumas métricas de desempenho. A principal delas é a acurácia, representando qual a capacidade do modelo em classificar corretamente as categorias (*True Positive* e *True Negative*)

$$Acc = \frac{TP + TN}{TP + TN + FN + FP} \quad (2.8)$$

A precisão (P), por sua vez, representa quantas categorias positivas foram feitas em relação ao total de categorias positivas. Importante ressaltar que essa precisão pode ser feita em relação às n classes presentes no modelo, como será visto nos Resultados.

$$P = \frac{TP}{TP + FP} \quad (2.9)$$

A sensibilidade (Sn) ou *Recall* representa a capacidade do modelo de acertar a real classe em razão das classificações feitas para ela.

$$Sn = \frac{TP}{TP + FN} \quad (2.10)$$

Por fim, o *F1-Score* representa uma métrica com objetivo de combinar as demais:

$$F1 = \frac{2 * P * Sn}{P + SN} \quad (2.11)$$

Alterar o ponto de corte, altera também os resultados das métricas acima. Sendo assim, é utilizado o diagrama da curva ROC (*Receiver Operation Curve*) e o indicador *AUC* (*Area Under Curve*) para determinar qual melhor ponto de corte para aquele modelo. A Figura 2.3 representa um exemplo de curva ROC, um classificador ideal teria todos os dados classificados de maneira correta levando a taxa de Positivos Verdadeiros para 1 e a taxa de Falsos positivos em 0.

A curva ROC demonstra como o classificador em questão se comporta para cada ponto de corte definido, o melhor ponto de corte é aquele que mais se aproxima geometricamente do ponto ideal sendo assim o potencial escolhido. Diferentes curvas representam diferentes classificadores, é possível notar que quanto mais próximo do ponto ideal maior é a área. A integral da função definida pelo modelo representa o indicador *AUC* que quanto mais próximo de 1 melhor será o desempenho do classificador.

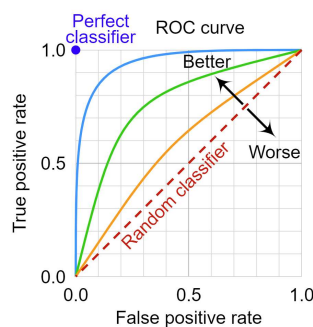


Figura 2.3: Exemplo de curva ROC com múltiplos classificadores. Fonte [Wikipedia](#)

A teoria por trás da classificação se propaga para qualquer modelo aplicado, assim como as métricas acima explicadas, a modificação entre eles será o algoritmo utilizado e o método de otimização do mesmo. As peculiaridades de cada modelo utilizado será discutido nas seções posteriores.

2.2.3 Regressão Logística Múltipla

A regressão Logística Múltipla é uma extrapolação para n_i classes para $i > 2$. Em teoria, trata-se de uma generalização do algoritmo apresentado acima.

Utilizando o mesmo conceito de *odds* acima, é possível determinar a *odds ratio* entre evento 1 e evento 2:

$$\text{odds ratio} = \frac{\pi_1}{\pi_2} \quad (2.12)$$

Onde a *odds ratio* representa a razão de chances entre duas classes. Com o *threshold* definido é possível determinar qual classe entre as n classes disponíveis o ponto observado se encontrará.

As diferenças na metrificação de desempenho para os modelos classificadores múltiplos é a presença de uma matriz de confusão 3x3, os dados classificados corretamente continuam se mantendo na diagonal principal da matriz com 3 opções mas o cálculo dos indicadores supracitados se mantém o mesmo.

2.2.4 K Vizinho Mais Próximos

O método dos K Vizinhos Mais Próximos se baseia na intuição de cálculo da distância de um ponto não visto do conjunto de teste com base nos dados segmentados para teste. A classificação pode ocorrer de algumas maneiras, vizinho mais próximo, média de k vizinhos mais próximos ou de maneira aleatória. Ainda, a distância entre pontos em espaços de n dimensões pode ser calculada de algumas maneiras: Euclidiana, Cossenos, Manhattan (ou *City Block*). A combinação de todos esses fatores geram resultados e acurácias diferentes a depender do contexto abordado. (KATARIA; SINGH, 2013)

Um exemplo do processo de classificação do método dos K Vizinhos é visto na Figura 2.4 onde o ponto P_i à ser classificado tem sua distância calculada com os pontos mais próximos. Uma maneira de classificar seria contar a média de pontos mais próximos, classificando-o como Classe II ou simplesmente igualá-lo pela menor distância, classificando-o como Classe III.

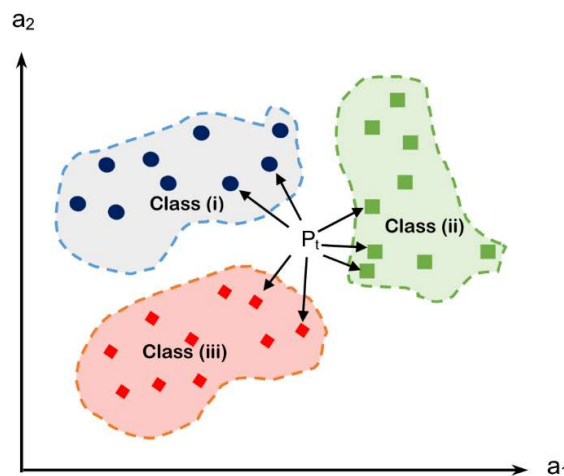


Figura 2.4: Exemplo para método de classificação com algoritmo dos K Vizinhos. Fonte: (ATALLAH; BADAWEY; EL-SAYED, 2019)

2.2.5 Árvores de Decisão e Florestas Aleatórias

O algoritmo de Árvore de Decisão foi proposto por Leo Breiman em seu livro *Classification and regression trees* em 1984. A intuição por trás do estudo de Breiman consiste na delimitação de valores que discriminem, considerando as variáveis observadas, regras de decisão para que um dado seja classificado dentro das opções. A Figura 2.5 representa uma estrutura de Árvore de Decisão para risco de vida de um paciente.

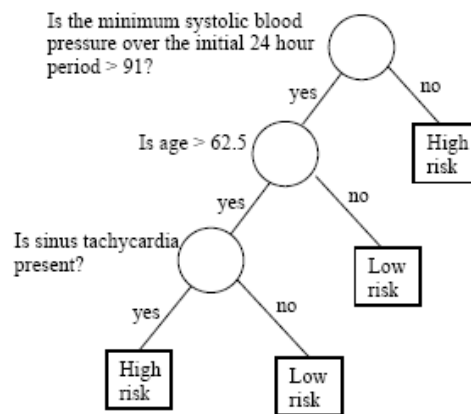


Figura 2.5: Exemplo de Árvore de Decisão

A Floresta Aleatória nada mais é que um conjunto agregado *ensemble* de algoritmos de classificação de árvores individual. As amostras são consideradas à depender da arquitetura do problema, os modelos mais comuns de amostragem para Árvores de Decisão são *Bagging* e *Boosting* podendo também ser combinados entre os dois. Cada modelo apresenta vantagens e desvantagens dependendo da natureza do problema.

O resultado final de uma Floresta Aleatória é medida após a construção de n árvores de decisão utilizando características e amostras aleatórias do conjunto de dados do problema. Por padrão das bibliotecas disponíveis, o resultado final é o resultado da média de cada uma das árvores ou, no caso da classificação, qual classe mais repetida.

2.2.6 Máquinas de vetores de suporte (SVM)

O SVM utiliza otimização para construir hiperplanos que separam as classes no espaço, visando maximizar o parâmetro "b" (margem), que representa a distância entre o hiperplano e o ponto mais próximo de cada classe, estabelecendo assim um limiar de decisão (HART; STORK; DUDA, 2000).

De maneira geral, para algoritmos *SVM* classificadores, existem dois tipos: lineares e não lineares. Ambos são muito sensíveis quanto às escalas das características no conjunto, sendo necessário um escalonamento em qualquer aplicação de Classificação *SVM*.

Os classificadores lineares são muito eficientes e performam bem em muitos casos de separações binárias, porém em questão de tempo de execução, funcionam melhor para conjuntos pequenos ou médios. Além disso, é comum encontrar casos que uma separação linear pura de duas classes é inviável, e uma das alternativas para solucionar essa situação, é, por exemplo, adicionar mais características, como as polinomiais. Ou seja, se adicionarmos $x_2 = x_1^2$, o conjunto de dados por se tornar linearmente separável. Porém, para conjuntos grandes, pode ocorrer uma explosão combinatória da quantidade de características (GÉRON, 2022).

Existem abordagens alternativas empregam funções matemáticas não lineares, chamadas *kernels*, que aumentam a dimensionalidade do espaço de características, melhorando a separação entre as classes, e evitam essa explosão combinatória de características. O *kernel* polinomial tende a classificar melhor com o aumento de sua ordem, porém pode gerar *overfitting* e aumentar o tempo de execução. Outra aplicação, é de uma função de similaridade ($\Phi_y(x, l)$), como a Função de Base Radial (*RBF*) Gaussiana (GÉRON, 2022).

$$\Phi_y(x, l) = \exp(y \|x - l\|^2) \quad (2.13)$$

Que busca medir o quanto cada instância (x) se assemelha a um ponto de referência específico (l).

Dentre os principais *kernels* existentes, são utilizados neste trabalho os seguintes: Polinomial (*POLY*), Sigmóide (*SIGMOID*) e a Função de Base Radial (*RBF*).

E quando se fala de complexidade computacional ao se utilizar de algoritmos que aceitam a aplicação do *kernel*, o período de treinamento tem complexidade geralmente entre $O(m^2 \times n)$ e $O(m^3 \times n)$, sendo m instâncias e n características. Ou seja, a performance do *SVM* diminui drasticamente com o aumento do número de instâncias.

2.2.7 Redes Neurais Artificiais - Perceptrons Multicamadas (*MLP*)

Ao longo da história, a espécie humana explorou correlações com a natureza para criarem as suas invenções. E quando fala-se de Redes Neurais Artificiais (*RNA*), a inspiração residiu no cérebro humano. No entanto, da mesma forma que aviões não precisam "bater as asas" como pássaros para voar, as *RNA* não precisam ficar restritas a sistemas biológicos plausíveis.

Dentre as RNA, o *Perceptron* tem uma das arquiteturas mais simples. Inventado em 1957 por Frank Rosenblatt, a estrutura do neurônio artificial segue a *unidade linear com threshold (LTU)*, no qual o *LTU* faz o cálculo da soma ponderada conforma a equação 2.14 de suas entradas, e posteriormente aplica uma função degrau ($step(z)$) a essa soma conforma a equação 2.15 (GÉRON, 2022).

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n = \mathbf{w}^T \cdot \mathbf{x} \quad (2.14)$$

Sendo w o peso aplicado a cada instância.

$$h_w(x) = step(z) = step(\mathbf{w}^T \cdot \mathbf{x}) \quad (2.15)$$

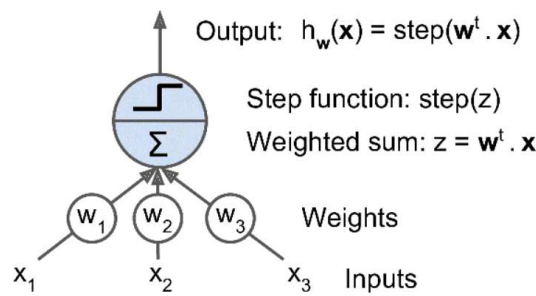


Figura 2.6: Unidade linear com Threshold (GÉRON, 2022).

As funções degrau mais comuns nos Perceptrons são:

$$heaviside(z) = \begin{cases} 0 & \text{se } z < 0 \\ 1 & \text{se } z \geq 0 \end{cases} \quad \text{e} \quad sgn(z) = \begin{cases} -1 & \text{se } z < 0 \\ 0 & \text{se } z = 0 \\ 1 & \text{se } z > 0 \end{cases} \quad (2.16)$$

Um Perceptron consiste em uma única camada de LTUs, onde cada neurônio está conectado a todas as entradas. O aprendizado do algoritmo segue uma variante da regra de Hebb, em que neurônios de saída que fazem previsões incorretas reforçam os pesos das conexões das entradas que contribuíram para a previsão correta, conforme a equação 2.17 de atualização de pesos (GÉRON, 2022).

$$w_{i,j}^{\text{próximo degrau}} = w_{i,j} + \eta(y_j - \hat{y}_j)x_i \quad (2.17)$$

Sendo $w_{i,j}$ o peso de conexão entre o i -ésimo neurônio de entrada e o j -ésimo neurônio de saída, x_i o valor da i -ésima entrada da instância de treinamento atual, $\hat{y}_j$ a j -ésima saída do neurônio de saída para a instância de treinamento atual, y_j a j -ésima saída alvo do neurônio de saída para a instância de treinamento atual e η é a taxa de treinamento.

Logo, fronteira de decisão de cada neurônio de saída é linear, o que significa que os Perceptrons não são capazes de aprender padrões complexos. A maneira de se eliminar boa parte dessas limitações foi empilhar vários Perceptrons, e essa RNA resultante é chamada de Perceptron Multicamada (*MLP*) (GÉRON, 2022). No entanto, foi somente na década de 80 que Rumelhart, Hinton e Williams (1986) comprovaram experimentalmente que a técnica da retropropagação (*backpropagation*) poderia ser uma solução viável para treinar redes multicamadas.

Um *MLP* é formado por uma camada de entrada (*passthrough*), uma ou mais camadas *LTUs* chamadas de camadas ocultas e uma camada final *LTU* denominada camada de saída. Cada camada, exceto a camada de saída, inclui um neurônio de viés e tem conexões completas com a camada subsequente, conforme a Figura 2.7.

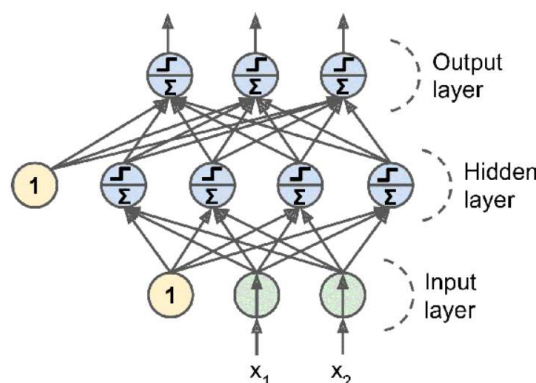


Figura 2.7: Um MLP moderno para classificação (incluindo ReLU e Softmax) (GÉRON, 2022).

Diferente do Perceptron, para cada instância de treinamento, o algoritmo de retropropagação da *MLP* faz uma previsão, mede o erro, e passa por cada camada no reverso para medir a contribuição do erro. O ajuste dos pesos da conexão para reduzir o erro é a etapa do Gradiente Descendente.

Como no Perceptron usa-se uma função degrau, não há gradiente para se trabalhar. Assim, a *MLP* usa as *funções de ativação*, sendo que as mais utilizadas são a tangente hiperbólica ($\tanh(z) = 2\sigma(2z) - 1$) e a ReLU ($\text{ReLU}(z) = \max(0, z)$) (GÉRON, 2022).

Redes neurais conforme a Figura 2.7, na qual o sinal flui em apenas uma direção (das entradas para as saídas), recebe a classificação de *rede neural feedforward* (*FNN*).

O Perceptron Multicamadas, ainda, tem o seu funcionamento validado pelo Teorema da aproximação universal, que diz que uma *MLP* consegue aproximar funções contínuas em um subconjunto compacto de $\mathbb{R}^n$ (CSÁJI et al., 2001).

2.3 Revisão Bibliográfica

O tema de detecção de falhas em conjunto de técnicas de Inteligência Artificial não é recente, apesar da relevância crescente do tema. Em 2003, Sharma et al. (2004) desenvolveu um trabalho sobre a aplicação de Redes Neurais utilizando uma arquitetura de 6-4-6 (seis neurônios de entrada, quatro em camada oculta e seis de saída) e uma arquitetura 3-4-6 para a detecção de seis tipos de falhas no processo de destilação de amônia em coluna de recheio.

O sistema foi treinado com dados de falha aplicando um distúrbio positivo e negativo de 5, 12.5 e 20 % nas variáveis: vazão de entrada, composição de entrada e saída de vapor. A rede neural, inicialmente na arquitetura 6-4-6 apresentou resultados bastante satisfatórios na predição de falhas em um sistema líquido-vapor simples. Seu Erro Quadrático Médio (*RMSE*) foi de 0.0212 enquanto o *MAPE* (*Maximum Average Percentage Error*) foi de 11% e o Coeficiente de Correlação de Pearson aproximadamente 0.99.

Ainda, a rede neural foi testada sob o contexto de duas e três falhas simultâneas ocorrerem para tentar simular um cenário mais verossímil onde erros podem se propagar em mais de um evento anômalo. Apesar da capacidade de generalização dos modelos de Rede Neural o autor conseguiu demonstrar somente a detecção de duas falhas ao mesmo tempo para alguns casos e nenhum caso da detecção simultânea de três.

Outros cenários no escopo da operação industrial também são investigados, o artigo de Andre, Beltrame e Wainer (2013) levanta a hipótese de combinar métodos de *Support Vector Machines* e KNN para detecção de falhas em elementos rotativos como bombas, esteiras de rolagem e engrenagens. Os atributos avaliados para a detecção foram anomalias referentes à velocidade, aceleração e envelope (transformação do sinal de aceleração para altas frequências).

O autor ainda discute o acoplamento de vantagens dos dois modelos: a capacidade de extrapolação dos fenômenos dado um ponto longe do hiperplano para o SVM e a estrutura de comparação de pontos pela distância do KNN em regiões muito próximas do hiperplano.

Os dados foram treinado com uma série histórica de 11 meses e testados com os valores do décimo segundo mês. Os resultados da combinação dos modelo demonstrou boa capacidade de

extrapolação dos dados, tendo uma acurácia média de 75% para motores de indução, 80% para elementos rotativos e 83% para acoplamento de engrenagem.

O artigo de [Pardo e Sberveglieri \(2008\)](#) discorre sobre a aplicação de Florestas Aleatórias e SVM no tratamento de dados de sensores além de um algoritmo de *Nearest Shrunk Centroids* (NSC). O autor relata que, apesar de potente, o NSC desempenha aquém dos algoritmos de Floresta Aleatória para conjuntos grandes de dados.

Capítulo 3

Metodologia

3.1 3W Dataset

Os dados utilizados para desenvolvimento foram reunidos no trabalho de [Vargas \(2019\)](#) através do estudo de um *dataset* referente ao processo de elevação da Unidade de Operação do Espírito Santo da Petrobras (UO-ES) acompanhado pelos especialistas e responsáveis pelo monitoramento da operações.

O *dataset* se encontra público no [GitHub](#) do autor e apresenta uma estrutura de 9 pastas. Sendo a de número 0 compilando dados de operação normal da Unidade de Operação, as unidades 1 a 8 sendo respectivamente:

- **Aumento de *Basic Sediment Water*:** Razão entre a vazão de água mais sedimentos produzidos por vazão de líquido produzido.
- **Fechamento espúrio da DSHV:** A *downhole safety valve* é a válvula de segurança instalada na coluna de produção do poço. Seu objetivo é o fechamento do poço em caso de desconexão física com a unidade de produção e o poço.
- **Intermitência Severa:** A intermitência severa dos equipamentos resulta em estresse mecânico indesejado.
- **Instabilidade no Fluxo:** Detecção, dentro das séries temporais, modificações relevantes porém toleráveis.

- **Perda Rápida de Produtividade:** A modificação das características do poço durante sua utilização. Tais modificações podem alavancar as perdas de carga fazendo com que o produto não atinja a superfície.
- **Restrição Rápida em CKP:** A válvula *Choke* de Produção é responsável pelo controle do poço na superfície.
- **Incrustação em CKP:** Há possibilidade de depósito de inorgânicos provenientes do poço, podendo impactar a produção total do poço.
- **Hidrato em Linha de Produção:** Hidratos são compostos cristalinos formados por água e gás natural. Sua desobstrução pode custar cerca de U\$ 500.000 dólares ao dia ([ANDREOLLI, 2016](#)).

Ainda, os dados estão dispostos na estrutura de 10 colunas: *timestamp*, P-PDG, P-TPT, T-TPT, P-MON-CKP, T-JUS-CKP, P-JUS-CKGL, T-JUS-CKGL, QGL, class. Sendo *timestamp* o registro do dia e horário da amostra, P-PDG *Permanent Downhole Gauge* indica a pressão no ponto mais próximo do reservatório, TPT indica a pressão e a temperatura do fluido que escoar pela Árvore de Natal Molhada, CKP representa a válvula *choke* de produção que possui a função de isolar o poço caso alguma desconexão ocorra com a plataforma, o sensor P-MON-CKP representa a pressão à montante da válvula enquanto T-JUS-CKP representa a temperatura à jusante da mesma. Já os dados de P-JUS-CKGL e T-JUS-CKGL representam a pressão e a temperatura à jusante da válvula *Choke de Gas Lift (CKGL)*, enquanto QGL representa a vazão do gás usado para elevação. Apenas unidades de engenharia foram consideradas para medição: Pascal [Pa], metros cúbicos por segundo padronizados [m^3/s] e graus Celsius [$^{\circ}\text{C}$].

Os dados utilizados no 3W possuem algumas vantagens como: vasta disponibilidade de informação, classes desbalanceadas dentro de uma mesma falha, estados de transição entre operação normal e falha, adição de dados simulados com intuito de adicionar dados de falhas menos recorrentes, diferentes tipos de falha e possuir dados bastante recentes a partir de 2017, além de ter um número de aplicações crescente no último ano que o utilizam como base, permitindo a comparação de resultados com demais trabalhos.

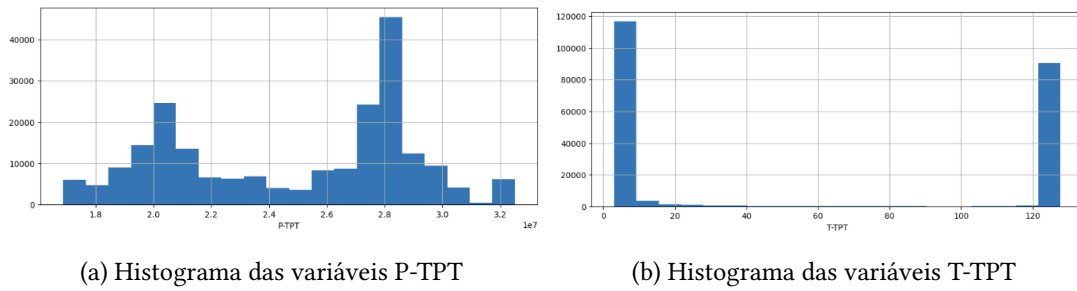


Figura 3.1: Histograma das variáveis T-TPT e P-TPT Fonte: Autores.

As falhas de tipo 3 e 4 não possuem estado Normal e Transiente de Falha para que a classificação seja aplicada, sendo desconsideradas na etapa de Classificação de Falhas.

3.2 Modelos de Classificação

A seguir os modelos serão comparados para cada tipo de falha presente no *dataset*. O intuito não é de realizar a otimização individual de cada um dos modelos mas explorar possibilidades de aplicação e confiabilidade dos resultados obtidos através das análises de Sensibilidade, Revocação, *Score F1*, Acurácia, Possibilidade de *Overfitting* e análises qualitativas da matriz de confusão obtida, todos os indicadores dos modelos foram considerados pela média ponderada das classes (*average = 'weighted'*).

Todos os códigos foram desenvolvidos em *Python 3.11.5* e os pacotes utilizados foram, principalmente: *SciKit Learn*, para modelos de Aprendizado de Máquina, *Numpy* para efetuar operações matemáticas, *Matplotlib*, *Seaborn* para construção dos gráficos e *Pandas* e *Polars* para manejo dos datasets e grandes volumes de dados. Os hiper parâmetros alterados serão descritos dentro da sub seção do modelo pertinente, todos os não citados seguiram o padrão definido na biblioteca. Para o treinamento foi dividido 30% dos dados para Treino e 70% para teste. Vale ressaltar que no caso dos algoritmos de *Support Vector Machines* e Redes Neurais, os *datasets* tiveram uma filtragem aleatória, na qual foram selecionados uma amostra equivalente à 10% dos dados originais, mantendo um valor de *seed* fixo para questões de reprodutibilidade. Essa consideração foi necessária devido a demanda computacional na etapa de treinamento dos modelos, pois como há uma limitação de *hardware* o tempo de execução do treinamento para o conjunto completo estava atingindo valores muito altos, o que inviabilizaria o treinamento destes modelos para todos os casos.

3.3 Aplicação

Com intuito de unir teoria e desenvolvimento de modelos com o mundo de aplicação de sistemas no monitoramento de anomalias em processos químicos foi construído uma interface *GUI* contendo as principais ferramentas desenvolvidas durante o trabalho. A aplicação concentra a análise individual das falhas, assim como *links* diretos para referências utilizadas, análise exploratória com gráficos de dispersão, histograma, *Box Plot* e a aplicação dos modelos de Aprendizado de Máquina através de interface interativa.

A aplicação foi construída como interface web através do *framework Flask*, utilizando linguagem Python para sustentação. Para o *front-end* e responsividade foi utilizado *Java Script* e para a diagramação e design das estruturas de dados foi utilizado *HTML*.

As páginas se dividem da seguinte maneira: Página Inicial, com uma breve introdução do projeto. Escolha de *dataset*, para o usuário determinar qual evento será analisado. Análise Exploratória, após a escolha do evento os gráficos são gerados em formato de carrossel. Resultados Modelos, com uma breve ilustração do modelo, seus hiperparâmetros utilizados, métricas de desempenho e matriz de confusão. Todos os códigos se encontram em Anexo e disponível no GitHub dos autores para utilização posterior.

Capítulo 4

Resultados e Discussão

4.1 Regressão Logística Múltipla

Os resultados do modelo de Regressão Logística Múltipla podem ser encontrados na Tabela 4.1. Em resumo, o modelo não se mostrou capaz de detectar bem as ocorrências de falha para os Eventos 1, 5, 7 e 8, retornando uma classificação com $F1 < 90\%$. Além disso, os tempos de execução para os Eventos 5 e 8 são significativamente altos dada a acurácia retornada. Além disso os Eventos 1 e 7 apresentam acurácia e F1 muito baixos, 30% e 61% para a acurácia e 36% e 46% para o F1.

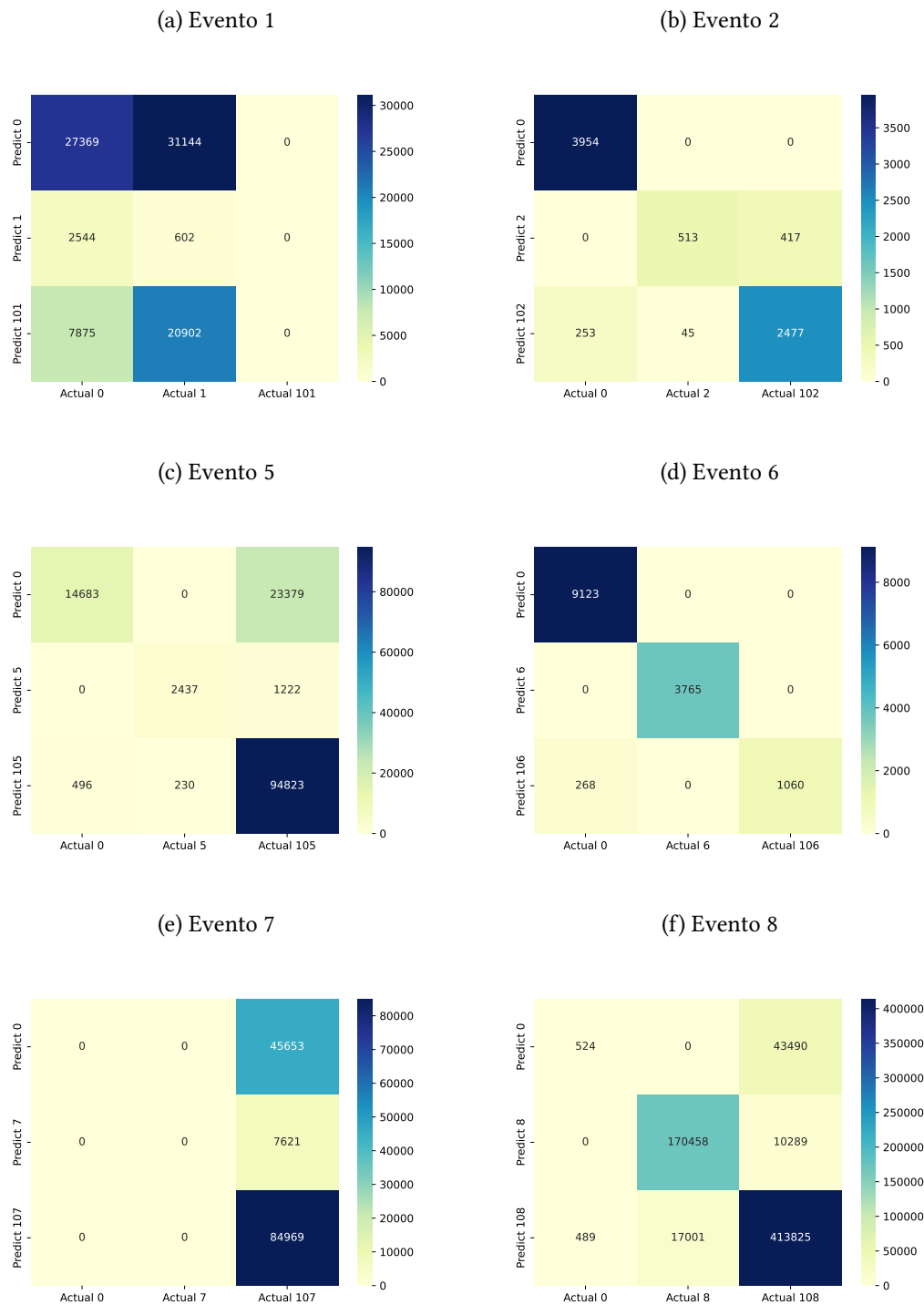
A análise da matriz de confusão (Figura 4.1 a 4.1) mostra que os Eventos 1 e 7 foram mal classificados. Para o Evento 1 o modelo não conseguiu prever nenhum estado transiente de falha corretamente (Actual 101) enquanto o Evento 7 retornou apenas estados transiente de falha, ignorando os eventos normal e falha.

Os hiperparâmetros utilizados no modelo foram: *multi_class* = 'multinomial' para classificações múltiplas, *solver* = 'newton-cg', *max_iter* = '10000', *class_weight* = None todos os outros seguiram a configuração *default* da biblioteca utilizada.

Tabela 4.1: Métricas de desempenho do modelo de Regressão Logística Múltipla para diferentes eventos.

Análise de métricas para Regressão Logística						
	Evento 1	Evento 2	Evento 5	Evento 6	Evento 7	Evento 8
Acurácia (%)	31,0	91,0	81,5	98,1	61,5	89,1
Sensibilidade (%)	31,0	91,0	81,5	98,1	61,5	89,1
F1	36,9	90,1	78,8	98,0	46,8	86,2
Tempo de Processamento (s)	1,2	31,5	184	8,0	1,2	889

Figura 4.1: Matriz de Confusão para cada tipo de Evento utilizando Regressão Logística Múltipla.



4.2 K Vizinhos Mais Próximos

O algoritmo dos K Vizinhos Mais próximos tem seus resultados registrados na Tabela 4.2. Em suma o modelo performou de maneira satisfatória em todos os casos, alcançando acurácia maior que 99% para os Eventos 2, 6,7,8, 96% para o Evento 1 e 93% para o Evento 3.

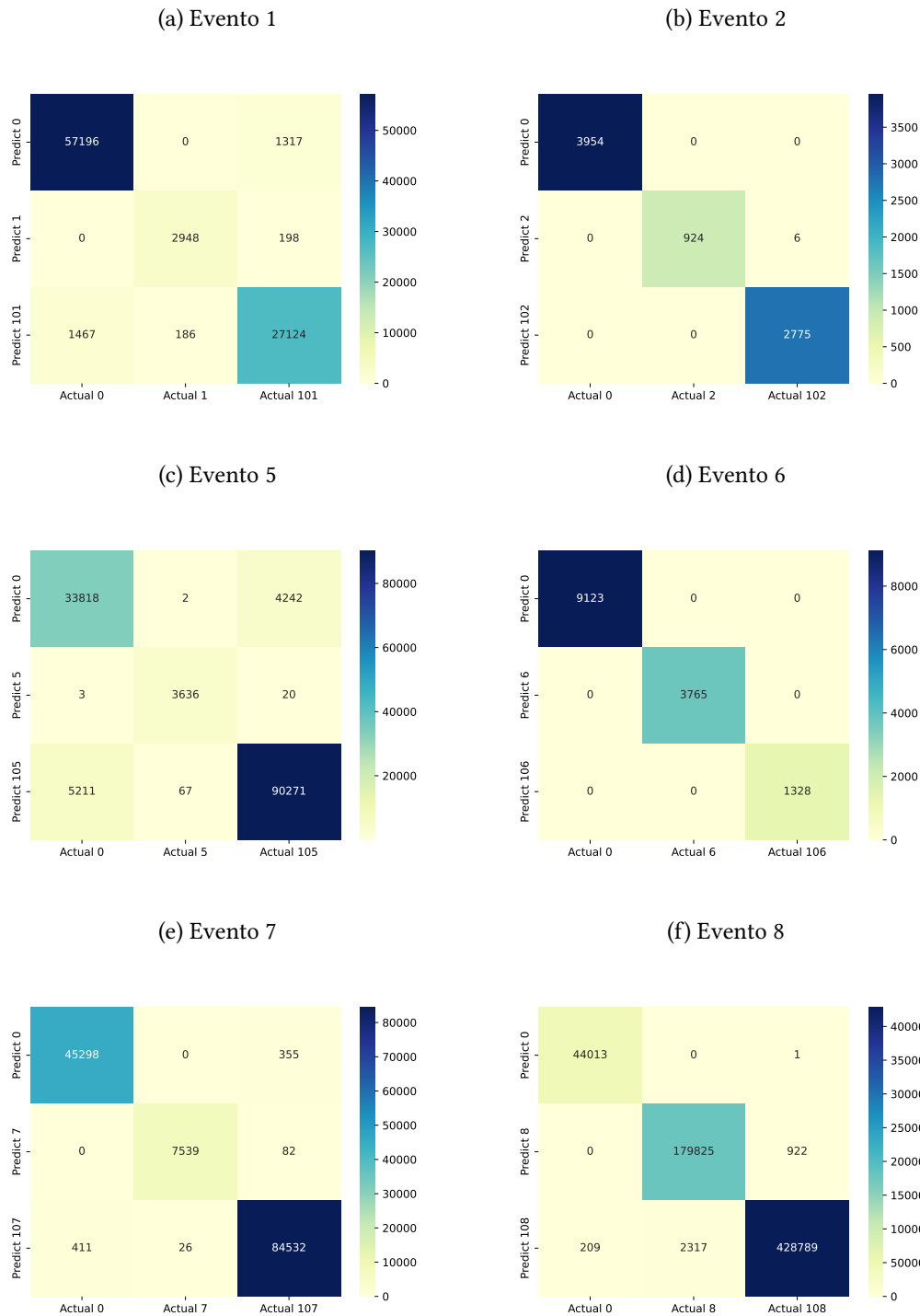
Sobre os hiperparâmetros utilizados, $n_neighbors = 5$, ou seja a média da distância dos 5 vizinhos mais próximos foi considerado, $weights = 'uniform'$, $algorithm = 'auto'$ e $leaf_size = 40$. Nenhum tipo de otimização com a função *GridSearchCV* foi realizado uma vez que a heurística inicial já retornava resultados considerados satisfatórios.

No quesito tempo de processamento, todos os algoritmos possuem uma resposta rápida (< 60 segundos) podendo ser considerados úteis em um ambiente real de produção. Sobre falhas críticas, apenas o Evento 5 possui eventos normais classificados como falha e vice versa, porém ainda em uma volumetria pouco impactante em comparação com outras classes. No mais, a maioria dos erros consiste entre Normal - Transiente de Falha e Transiente de Falha - Falha.

Tabela 4.2: Métricas de desempenho do modelo de K Vizinhos Mais Próximos para diferentes eventos.

Análise de métricas para K Vizinhos Mais Próximos						
	Evento 1	Evento 2	Evento 5	Evento 6	Evento 7	Evento 8
Acurácia (%)	96,5	99,9	93,0	100	99,4	99,5
Sensibilidade (%)	96,5	99,9	93,0	100	99,4	99,5
F1	96,5	99,9	93,1	100	99,4	99,5
Tempo de Processamento (s)	13,9	0,5	4,3	0,4	16,2	36,8

Figura 4.2: Matriz de Confusão para cada tipo de Evento utilizando K Vizinhos Mais Próximos.



4.3 Floresta Aleatória

O algoritmo de Floresta Aleatória tem seus resultados registrados na Tabela 4.3. Em suma o modelo performou de maneira satisfatória em todos os casos, alcançando acurácia maior que

93% para todos os casos e conseguindo performar com 100% de acurácia para os Eventos 2, 6 e 7. O Evento 5 teve a pior performance porém ainda ficando em um patamar satisfatório quando comparado com algoritmos mais robustos (*MLP*).

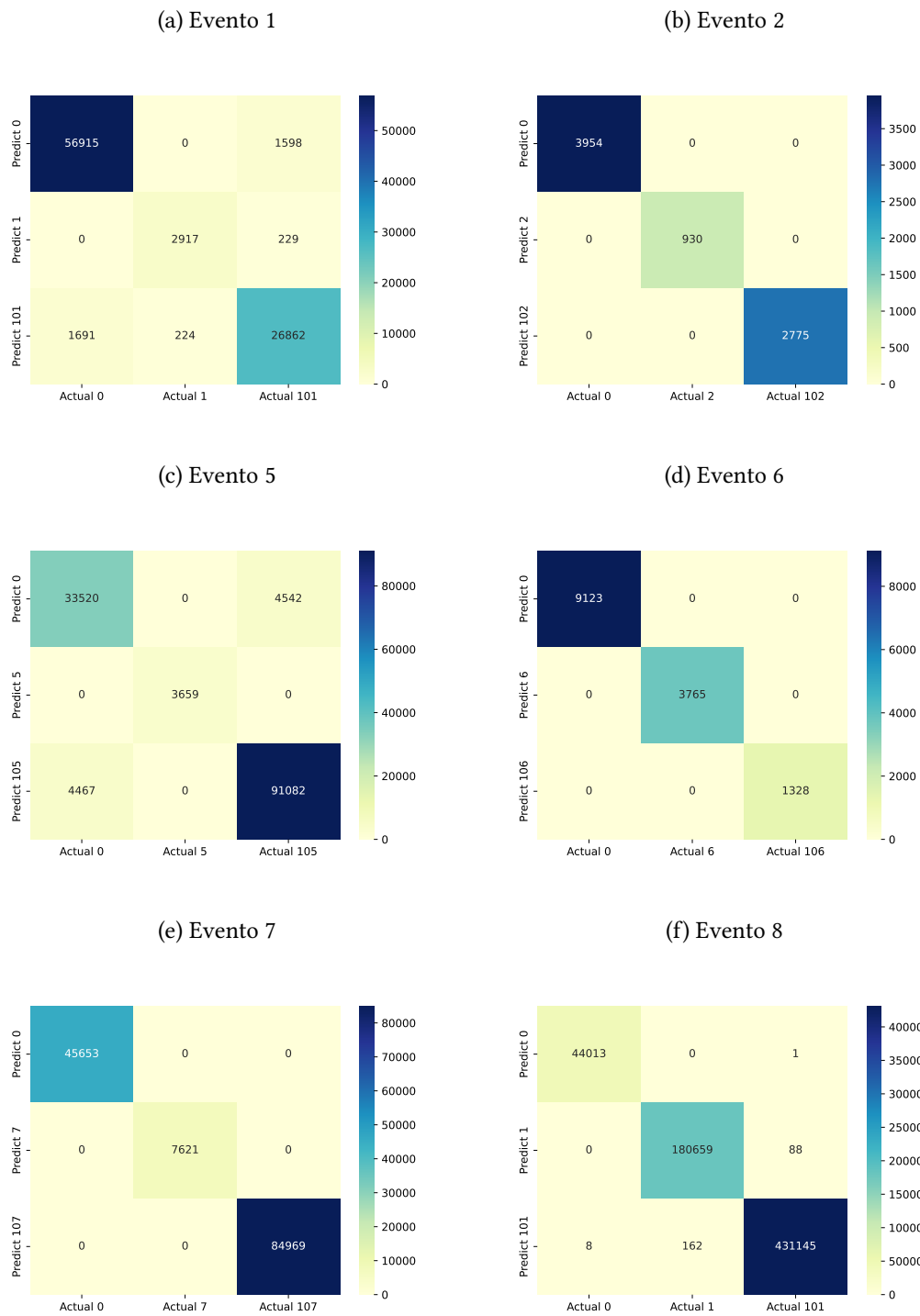
Ainda, sobre a questão de processamento apresenta um custo benefício interessante entre tempo e acurácia tendo apenas os Eventos 5, 7 e 8 com tempo de processamento maior que 60 segundos. Apesar de ser considerado um tempo mais demorado em comparação ao Evento 2, com 1,2 segundos de processamento, o algoritmo de Floresta Aleatória no Evento 5 e 7 ainda pode ser considerado um tempo de processamento hábil para aplicação prática. Já para o Evento 8 seria necessário a avaliação mais minuciosa para reduzir questões de processamento, visto que houve um retorno de acurácia da classificação em 99,96%.

Os parâmetros utilizados foram *criterion* = 'gini' para medição da qualidade da partição feita na classificação da folha da árvore e *oob_score* = 'True' para utilizar amostras *out of bag* na validação do modelo.

Tabela 4.3: Métricas de desempenho do modelo de Floresta Aleatória para diferentes eventos.

Análise de métricas para Floresta Aleatória						
	Evento 1	Evento 2	Evento 5	Evento 6	Evento 7	Evento 8
Acurácia (%)	95,9	100	93,4	100	100	99,9
Sensibilidade (%)	95,9	100	93,4	100	100	99,9
F1	95,9	100	93,4	100	100	99,9
Tempo de Processamento (s)	23,6	1,2	120	6,6	66,9	789

Figura 4.3: Matriz de Confusão para cada tipo de Evento utilizando Floresta Aleatória.



4.4 Máquinas de vetores de suporte (SVM)

O classificador *SVM* obteve bons resultados de acurácia para os modelos, conforme a Tabela 4.4, ficando acima de 85% para a maioria dos casos, com exceção do Evento 1, que obteve 79% de acurácia. Os parâmetros de sensibilidade e F1-score seguem a mesma tendência.

Tabela 4.4: Métricas de desempenho do modelo de Máquinas de vetores de suporte para diferentes eventos.

Análise de métricas para Máquinas de vetores de suporte						
	Evento 1	Evento 2	Evento 5	Evento 6	Evento 7	Evento 8
Acurácia (%)	79,6	99,0	85,2	95,5	86,3	89,8
Sensibilidade (%)	79,6	99,0	85,2	95,5	86,3	89,8
F1	79,0	99,0	82,1	95,3	83,8	86,8
Tempo de Processamento (s)	24291,0	3,0	1917,1	9,4	4102,0	472,9

O modelo utilizado foi a classe *SVC* da biblioteca *libsvm*, que implementa um algoritmo que suporta o truque do *kernel*. Os hiperparâmetros escolhidos foram $C = 10$, $kernel = 'rbf'$, $gamma = 0.3$ e $degree = 3$.

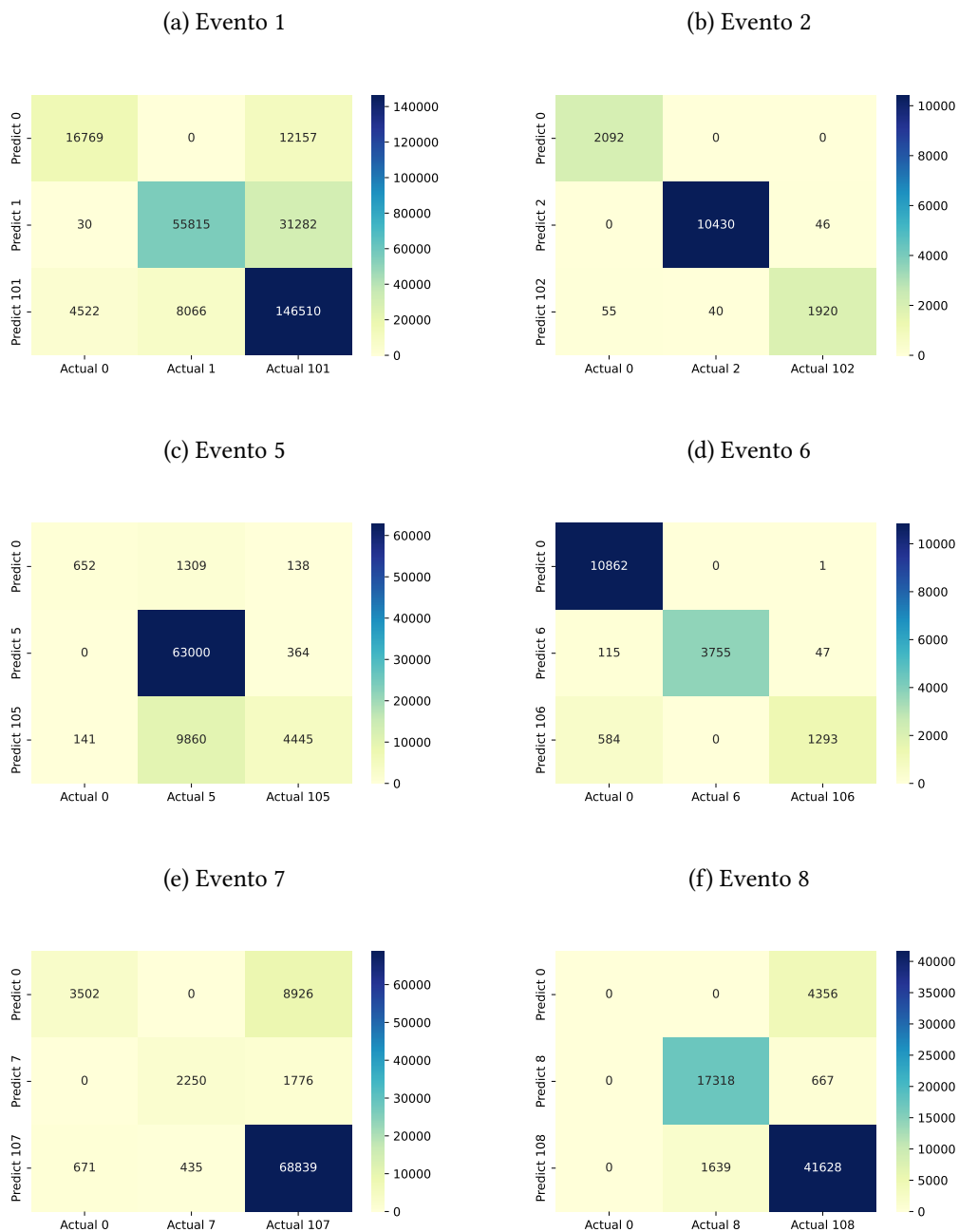
Com base na análise da matriz de confusão (Figuras 4.4 a 4.4), observa-se que o desempenho do *SVM* é ligeiramente inferior em comparação com outros modelos. No entanto, sua capacidade de resolver problemas complexos evitou, na maioria das vezes, a ocorrência do evento crítico, ou seja, quando um estado normal é classificado como falha e vice-versa. A exceção ocorre na anomalia 5, que teve um número considerável de classificações nesse evento crítico.

O *kernel* RBF Gaussiano obteve resultados melhores do que o polinomial, sigmoid e o linear. O que faz sentido pelo tipo de problema abordado, de tal forma que conjuntos complexos conseguem ser bem classificados, pois o kernel consegue simular a adição de muitas características de similaridade sem de fato adicioná-las ao modelo.

Em relação ao tempo de execução, o *SVM* teve os maiores tempos de treinamento e de teste, mesmo levando em consideração a redução amostral dos conjuntos de cada evento. Isso corrobora com a literatura, pois os conjuntos de dados de cada anomalia são grandes, e um maior número de instâncias prejudica o tempo de treinamento do algoritmo. A discrepância

é evidente ao comparar as anomalias 1 e 2, já que o *dataset 1* é mais de 10 vezes maior que o *dataset 2*. Como resultado, o treinamento do *dataset 1* levou cerca de 7 horas, enquanto o *dataset 2* foi treinado em apenas 3 segundos.

Figura 4.4: Matriz de Confusão para cada tipo de Evento utilizando Máquinas de vetores de suporte.



4.5 Perceptron Multicamadas - Redes Neurais Artificiais

O Perceptron Multicamadas aplicado tem os seus resultados registrados na Tabela 4.5. Em resumo, o desempenho do modelo se mostrou alto, registrando uma acurácia maior que 90% para todos os eventos. O mesmo vale para os parâmetros de Sensibilidade e F1-score, com exceção do Evento 8, que teve um F1-score de 89%.

Tabela 4.5: Métricas de desempenho do modelo de Perceptron Multicamadas para diferentes eventos.

Análise de métricas para Perceptron Multicamadas						
	Evento 1	Evento 2	Evento 5	Evento 6	Evento 7	Evento 8
Acurácia (%)	90,5	99,3	93,0	96,8	93,7	90,9
Sensibilidade (%)	90,5	99,3	93,0	96,8	93,7	90,9
F1	90,5	99,3	92,5	96,7	93,7	89,1360
Tempo de Processamento (s)	1270,4	25,9	21070,4	10,9	249,9	184,0

A análise da matriz de confusão (Figura 4.5 a 4.5) nos mostra que as anomalias 1, 2, 6, 7 e 8 tiveram nenhum ou poucos eventos crítico classificados de forma errada. A anomalia 5, no entanto, teve uma quantidade considerável de dados que na realidade eram uma falha, porém foram classificados como estado normal de operação. A maior parcela de erros está associada entre o estado transiente e o estado de falha ou o estado transiente e estado normal.

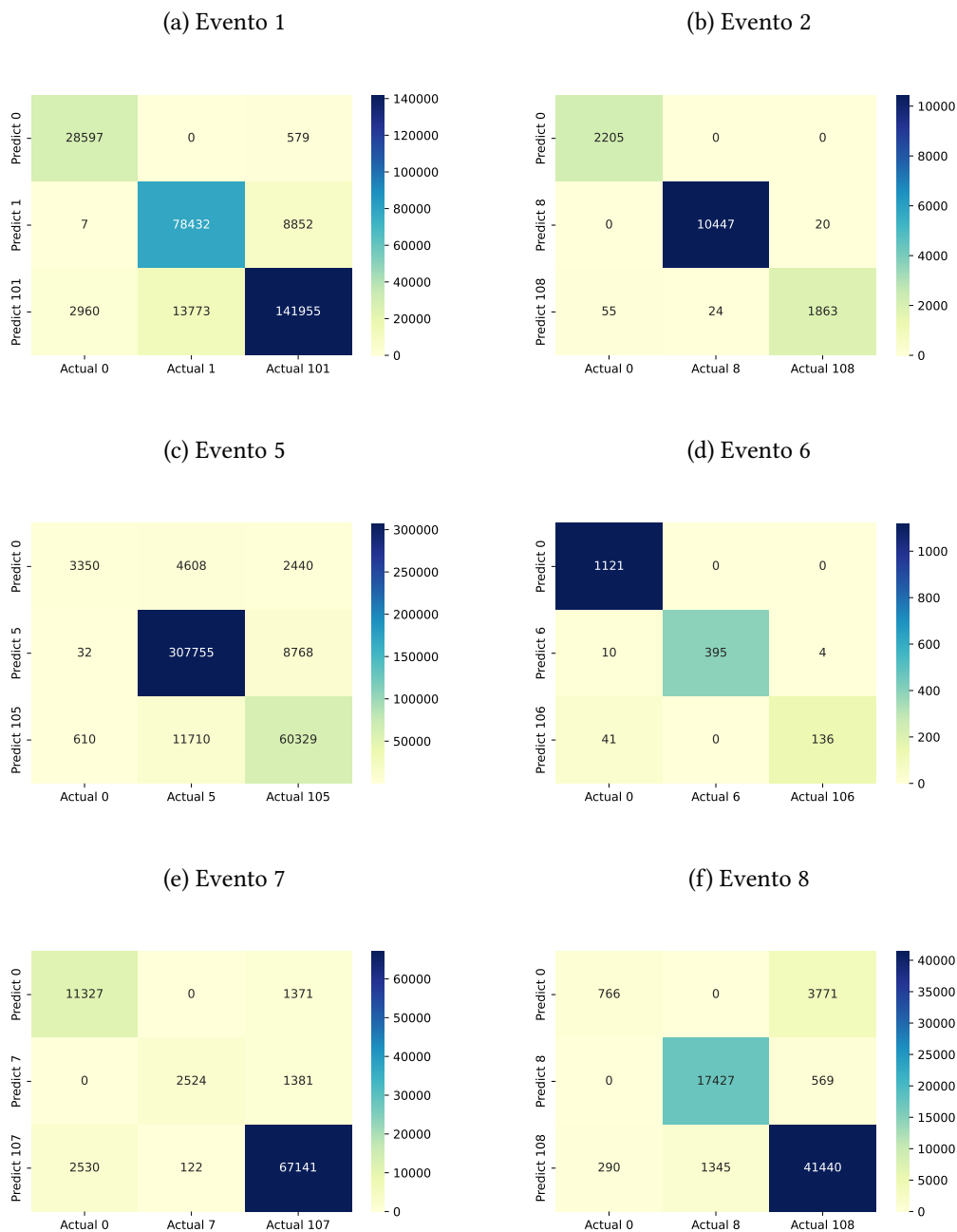
Os hiperparâmetros utilizados no modelo foram: *hidden_layer_sizes* = (20,20) duas camadas ocultas com 20 neurônios cada, *activation* = 'relu', *solver* = 'adam', *alpha* = 0.0001, *random_state* = None e *max_iter* = 1000.

A rede neural teve seus parâmetros escolhidos após aplicação da função *GridSearchCV*, que faz uma busca em grade com validação cruzada sobre uma grade de parâmetros. Os hiperparâmetros não estão otimizados para cada evento, mas de maneira geral, performam bem para todos os casos.

Quanto ao número de camadas ocultas e o número de neurônios, seguiu-se uma abordagem de sobreajuste, fazendo a busca em grade com uma e duas camadas ocultas, e reduzindo a quantidade de neurônios até atingir um resultado de acurácia satisfatório. Essa quantidade de

camadas já atende o Teorema da aproximação universal, e é comumente utilizada em problemas classificatórios com três classes.

Figura 4.5: Matriz de Confusão para cada tipo de Evento utilizando Máquinas de vetores de suporte.



4.6 Aplicação DFPSE - Detecção de Falhas em Poços e Sistemas de Elevação

Visando o desenvolvimento da aplicação física dos modelos acima descritos e testados, foi desenvolvida a plataforma nomeada de *Detecção de Falhas em Poços e Sistemas de Elevação (DFPSE)*. A plataforma foi desenvolvida utilizando o framework *Flask* da linguagem *Python*. Um exemplo da Página Inicial e estrutura da aplicação pode ser visto à seguir:

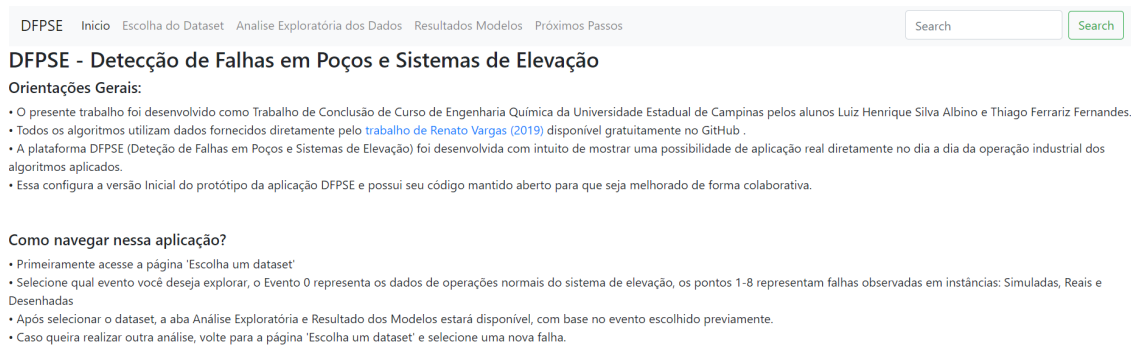


Figura 4.6: Visão da página inicial da Aplicação DFPSE.

A estrutura da aplicação segue na arquitetura de páginas dedicadas para cada funcionalidade da mesma. A Página de Início resume o intuito do desenvolvimento e o escopo em que foi desenvolvida. A aba Escolha do Dataset permite o usuário escolher qual evento será investigado dentro de uma caixa suspensa de escolha.

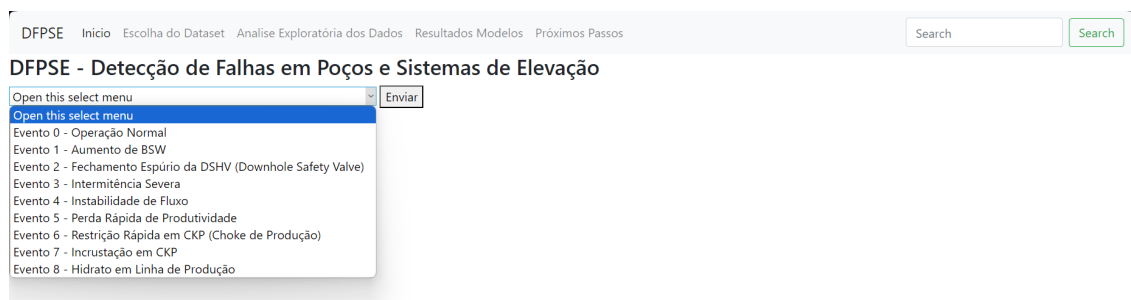


Figura 4.7: Visão da página de escolha do *dataset* da Aplicação DFPSE.

Assim, que o evento é escolhido o usuário consegue navegar para a página de Análise Exploratória e ter acesso a dados iniciais e estatísticas descritivas do evento escolhido. O usuário consegue trafegar pelos dados através de uma estrutura de gráficos pré escolhidos para análise da dispersão dos dados, gráficos no estilo *BoxPlot* e Histograma de Dispersão estão disponíveis para visualização.

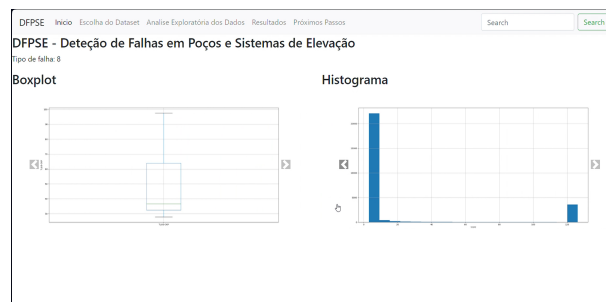


Figura 4.8: Visão da página de análise exploratória do *dataset* da Aplicação DFPSE.

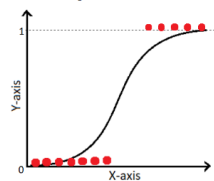
Com o evento selecionado, o usuário pode transitar para os resultados dos modelos. A ideia é que o usuário veja a comparação entre os cinco modelos para um mesmo Evento selecionado.

Ainda, a página oferece uma imagem explicativa de qual modelo está sendo utilizada, tal como sua matriz de confusão, parâmetros utilizados e indicadores de desempenho.

DFPSE - Detecção de Falhas em Poços e Sistemas de Elevação

Tipo de falha: 2

Regressão Logística Múltipla



Hiperparâmetros utilizados

multi_class: multinomial
solver: newton-cg
max_iter: 10000

Resultados e Métricas Matriz de Confusão

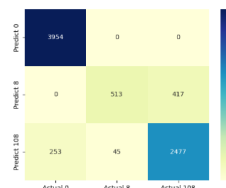
Acurácia Geral do modelo: 0.9066

F1 Score Geral do modelo: 0.9006

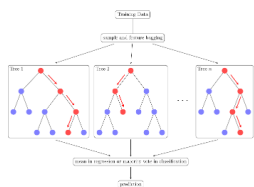
Recall do modelo para Classe 0 (Normal): 1.0000

Recall do modelo para Classe 102 (Transiente): 0.5516

Recall do modelo para Classe 2 (Falha): 0.8926



Random Forest



Hiperparâmetros utilizados

criterion: gini
oob_score: True

Resultados e Métricas

Acurácia Geral do modelo: 1.0000

F1 Score Geral do modelo: 1.0000

Recall do modelo para Classe 0 (Normal): 1.0000

Recall do modelo para Classe 102 (Transiente): 1.0000

Matriz de Confusão

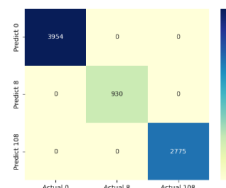


Figura 4.9: Visão dos resultados dos modelos para o Evento 2.

Assim, é possível implementar uma camada de abstração entre o usuário final e a camada de código, facilitando a democratização das ferramentas para Engenheiros de Processos, de Petróleo ou outros usuários de interesse que, por ventura, não possuem familiaridade com linhas de comando e programação. Ainda, o processo de desenvolvimento da aplicação facilita a contribuição de diferentes frentes dentro do produto final, agregando áreas de conhecimento diversas na contribuição e construção de um *software* robusto e de vasta aplicação.

Por fim, a aplicação não foi construída em um servidor externo e robusto aberto para acessos externos. Para fins de teste da aplicação os *scripts* foram utilizados apenas na máquina local

(*localhost*) acessando através do IP da máquina Local. Todos os códigos desenvolvidos tanto para os modelos quanto para a aplicação final estarão disponíveis em Anexo e no *GitHub* como forma de fomentar a utilização do *dataset* 3W e de modelos de Aprendizado de Máquina para outros futuros trabalhos.

Capítulo 5

Conclusão

Considerando todos os modelos propostos analisados, é possível considerar que modelos de Aprendizado de Máquina conseguem classificar de maneira satisfatória todas as falhas analisadas presentes na base de dados 3W disponibilizada pela Petrobras.

Considerando uma média das acurácias e Pontuação F1 para cada um dos modelos é possível encontrar valores de 75% e 67% para a Regressão Logística Multinível; 98% e 98% para o K Vizinhos Mais Próximos; 98% e 98% para Floresta Aleatória; 93% e 93% para o Perceptron Multicamadas e 89 % e 87% para o SVM. No geral, apenas o modelo de Regressão Logística não foi capaz de generalizar os eventos de falha dentro dos diferentes eventos observados, todos os outros possuem uma acurácia interessante para utilização em um processo de tamanha complexidade e custo de manutenção.

Os algoritmos de Regressão Logística Multinível e K Vizinhos Mais Próximos apresentaram classificação do tipo falha crítica. Entretanto, o modelo dos K Vizinhos tiveram apenas 3 casos para o Evento 5 enquanto a Regressão apresentou 33.688 casos para o Evento 1. É possível que, com uma investigação mais minuciosa dos hiper parâmetros o modelo dos K Vizinhos performe sem classificações críticas de falha.

No tocante ao tempo de execução, o algoritmo de SVM apresentou os maiores tempos, tendo casos com aproximadamente 7 horas de execução inviabilizando-o para aplicações em ambientes reais que exigem respostas dinâmicas e céleres à operação. Na média, o menor tempo de processamento é o do algoritmo dos K Vizinhos com um valor de 12 segundos sendo que o limite superior foi para o Evento 8, demorando 36,8 segundos de processamento.

Em suma, os algoritmos de K Vizinhos Mais Próximos, Floresta Aleatória e Rede Neural apresentaram acurácia satisfatória. Porém, considerando outros fatores como tempo de proces-

samento e falhas críticas seria indicado a consideração do algoritmo de Floresta Aleatória para uma aplicação prática *in loco* de detecção de falhas ou uma exploração mais minuciosa para otimização de hiperparâmetros do algoritmo dos K Vizinhos.

No quesito da aplicação, desenvolvimentos futuros para melhoria da responsividade, dinâmica, velocidade, integração da aplicação com sensores das linhas de elevação podem ser estudadas para aprimoramento contínuo. As sugestões comentadas nesta seção estão também compiladas na página "Próximos Passos".

Sugere-se como próximos passos a investigação da criação de um sensor virtual que consiga, além de discernir sobre o estado de falha ou não falha, discernir também qual tipo de Evento estaria ocorrendo. Uma vez que, na operação diária de um sistema de elevação, o tipo de Evento não é informado *a priori* da análise de um Especialista do setor. Assim, a aplicação teria ainda mais vantagens e poderia tratar de maneira prognóstica qual evento está ocorrendo durante a operação, agilizando os esforços de correção ou permitindo o planejamento de paradas preventivas dos equipamentos envolvidos.

Referências bibliográficas

- ALDRICH, C.; AURET, L. **Unsupervised process monitoring and fault diagnosis with machine learning methods**. [S.l.]: Springer, 2013. v. 16.
- ANDRE, A. B.; BELTRAME, E.; WAINER, J. A combination of support vector machine and k-nearest neighbors for machine fault detection. **Applied Artificial Intelligence**, Taylor & Francis, v. 27, n. 1, p. 36–49, 2013.
- ANDREOLLI, I. Introdução à elevação e escoamento monofásico e multifásico de petróleo. **Rio de Janeiro: Interciência**, 2016.
- ATALLAH, D. M.; BADAWEY, M.; EL-SAYED, A. Intelligent feature selection with modified K-nearest neighbor for kidney transplantation prediction. **SN Applied Sciences**, Springer, v. 1, p. 1–17, 2019.
- BARROS FILHO, J. **Ajuste automático e otimização do tempo de espera (Idle Time) em Bombeio Mecânico de Petróleo**. 2002. Diss. (Mestrado) – Universidade Federal do Rio Grande do Norte.
- BRO, R.; SMILDE, A. K. Principal component analysis. **Analytical methods**, Royal Society of Chemistry, v. 6, n. 9, p. 2812–2831, 2014.
- CHEUNG, J.-Y.; STEPHANOPOULOS, G. Representation of process trends—Part I. A formal representation framework. **Computers & Chemical Engineering**, v. 14, n. 4, p. 495–510, 1990. ISSN 0098-1354. DOI: [https://doi.org/10.1016/0098-1354\(90\)87023-I](https://doi.org/10.1016/0098-1354(90)87023-I). Disponível em: <<https://www.sciencedirect.com/science/article/pii/009813549087023I>>.
- COX, D. R. The Regression Analysis of Binary Sequences. **Journal of the Royal Statistical Society: Series B (Methodological)**, v. 20, n. 2, p. 215–232, dez. 1958. ISSN 0035-9246. DOI: [10.1111/j.2517-6161.1958.tb00292.x](https://doi.org/10.1111/j.2517-6161.1958.tb00292.x). eprint: https://academic.oup.com/jrsssb/article-pdf/20/2/215/49096552/jrsssb_20_2_215.pdf. Disponível em: <<https://doi.org/10.1111/j.2517-6161.1958.tb00292.x>>.
- CSÁJI, B. C. et al. Approximation with artificial neural networks. **Faculty of Sciences, Eötvös Loránd University, Hungary**, v. 24, n. 48, p. 7, 2001.
- FACELI, K. et al. Inteligência artificial: uma abordagem de aprendizado de máquina, 2021.
- GALEAZZI, A. et al. A Cloud Computing Application for the Supercritical Carbon Dioxide Extraction Using Coffee Grounds Silverskin. In: TÜRKAY, M.; GANI, R. (Ed.). **31st European Symposium on Computer Aided Process Engineering**. [S.l.]: Elsevier, 2021. v. 50. (Computer Aided Chemical Engineering). P. 1035–1040. DOI: <https://doi.org/10.1016/B978->

0-323-88506-5. 50159-5. Disponível em: <<https://www.sciencedirect.com/science/article/pii/B9780323885065501595>>.

GÉRON, A. **Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow**. [S.l.]: "O'Reilly Media, Inc.", 2022.

HART, P. E.; STORK, D. G.; DUDA, R. O. **Pattern classification**. [S.l.]: Wiley Hoboken, 2000.

HELBING, G.; RITTER, M. Deep Learning for fault detection in wind turbines. **Renewable and Sustainable Energy Reviews**, v. 98, p. 189–198, 2018. ISSN 1364-0321. DOI: <https://doi.org/10.1016/j.rser.2018.09.012>. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S1364032118306610>>.

KATARIA, A.; SINGH, M. A review of data classification using k-nearest neighbour algorithm. **International Journal of Emerging Technology and Advanced Engineering**, v. 3, n. 6, p. 354–360, 2013.

KULKARNI, A.; CHONG, D.; BATARSEH, F. A. Foundations of data imbalance and solutions for a data democracy. In: DATA democracy. [S.l.]: Elsevier, 2020. P. 83–106.

KUMAMOTO, H.; IKENCHI, K.; INOUE, K.; HENLEY, E. J. Application of expert system techniques to fault diagnosis. **The Chemical Engineering Journal**, Elsevier, v. 29, n. 1, p. 1–9, 1984.

MOMM, G. G. Detecção de anomalias em sensores de poços submarinos com uso de redes neurais artificiais.

NASCIMENTO, J. M. A. d. **Simulador computacional para poços de petróleo com método de elevação artificial por bombeio mecânico**. 2005. Diss. (Mestrado) – Universidade Federal do Rio Grande do Norte.

PARDO, M.; SBERVEGLIERI, G. Random forests and nearest shrunken centroids for the classification of sensor array data. **Sensors and Actuators B: Chemical**, v. 131, n. 1, p. 93–99, 2008. Special Issue: Selected Papers from the 12th International Symposium on Olfaction and Electronic Noses. ISSN 0925-4005. DOI: <https://doi.org/10.1016/j.snb.2007.12.015>. Disponível em: <<https://www.sciencedirect.com/science/article/pii/S0925400507009914>>.

SANTOS, M.; MANHÃES, A. M.; LIMA, A. R. Indústria 4.0: Desafios e oportunidades para o Brasil. **Anais do X SIMPROD**, Departamento de Engenharia de Produção-Universidade Federal de Sergipe, 2018.

SHARMA, R.; SINGH, K.; SINGHAL, D.; GHOSH, R. Neural network applications for detecting process faults in packed towers. **Chemical engineering and processing: process intensification**, Elsevier, v. 43, n. 7, p. 841–847, 2004.

SUTTON, I. S. Summarizing the deepwater horizon/macondo reports. In: OTC. OFFSHORE technology conference. [S.l.: s.n.], 2013. otc-24027.

TRIGUEROS-PRECIADO, S.; PÉREZ-GONZÁLEZ, D.; SOLANA-GONZÁLEZ, P. Cloud computing in industrial SMEs: Identification of the barriers to its adoption and effects of its application. **Electronic Markets**, Springer, v. 23, p. 105–114, 2013.

VARGAS, R. **Base de Dados e Benchmarks para Prognóstico de Anomalias em Sistemas de Elevação de Petróleo**. 2019. Tese (Doutorado) – Tese de doutorado, Universidade Federal do Espírito Santo.

VENKATASUBRAMANIAN, V.; RENGASWAMY, R.; KAVURI, S. N.; YIN, K. A review of process fault detection and diagnosis: Part III: Process history based methods. **Computers & chemical engineering**, Elsevier, v. 27, n. 3, p. 327–346, 2003.

VENKATASUBRAMANIAN, V.; RENGASWAMY, R.; YIN, K.; KAVURI, S. N. A review of process fault detection and diagnosis: Part I: Quantitative model-based methods. **Computers & chemical engineering**, Elsevier, v. 27, n. 3, p. 293–311, 2003.

Apêndice A

Primeiro Apêndice

A.1 Modelos de Aprendizado de Máquina e Indicadores de Desempenho

```
def multi_log_regress(hiperparams, X_treino, Y_treino):  
    model = LogisticRegression(  
        multi_class = hiperparams[0]  
        , solver = hiperparams[1]  
        , max_iter = hiperparams[2]  
        , class_weight = None  
    )  
  
    model.fit(X_treino, Y_treino)  
  
    return model  
  
def multi_log_class(model, X_test, y_test):  
    #global multi_log_metrics  
    y_pred = model.predict(X_test)  
  
    accuracy_multi_log = accuracy_score(y_test, y_pred)  
    recall_multi_log = recall_score(y_test, y_pred, average = None)  
    f1_multi_log = f1_score(y_test, y_pred, average='weighted')  
  
    multi_log_metrics = [accuracy_multi_log, recall_multi_log, f1_multi_log]
```

```
cm = confusion_matrix(y_test, y_pred)
cm_matrix = pd.DataFrame(data=cm, columns=['Actual 0', 'Actual 8', 'Actual 108'],
                        index=['Predict 0', 'Predict 8', 'Predict 108'])
sns.heatmap(cm_matrix, annot=True, fmt='d', cmap='YlGnBu')

plt.savefig('static/images/heatmap_multilog.png')
return multi_log_metrics
```

```
def svm (hiperparams, X_treino, Y_treino):
```

```
    model = SVC(
        C = hiperparams[0] #10
        , kernel= hiperparams[1] #'rbg'
        , gamma = hiperparams[2] #0.3
        , degree= hiperparams[3] # 3
    )
```

```
    model.fit(X_treino, Y_treino)
```

```
    return model
```

```
def svm_class(model, X_test, y_test):
```

```
    #global multi_log_metrics
```

```
    y_pred = model.predict(X_test)
```

```
    accuracy_multi_log = accuracy_score(y_test, y_pred)
```

```
    recall_multi_log = recall_score(y_test, y_pred, average = None)
```

```
    f1_multi_log = f1_score(y_test,y_pred, average='weighted')
```

```
    multi_log_metrics = [accuracy_multi_log, recall_multi_log, f1_multi_log]
```

```
    cm = confusion_matrix(y_test, y_pred)
```

```
    cm_matrix = pd.DataFrame(data=cm, columns=['Actual 0', 'Actual 8', 'Actual 108'],
                        index=['Predict 0', 'Predict 8', 'Predict 108'])
```

```
    sns.heatmap(cm_matrix, annot=True, fmt='d', cmap='YlGnBu')
```

```
    plt.savefig('static/images/heatmap_svm.png')
```

```
    return multi_log_metrics
```

```
def random_forest(hiperparams, X_treino, Y_treino):
```

```
model = RandomForestClassifier(
    criterion = hiperparams[0]
    , oob_score = hiperparams[1]
)

model.fit(X_treino, Y_treino)
return model

def random_forest_class(model, X_test, y_test):
    #global multi_log_metrics
    y_pred = model.predict(X_test)

    accuracy_multi_log = accuracy_score(y_test, y_pred)
    recall_multi_log = recall_score(y_test, y_pred, average = None)
    f1_multi_log = f1_score(y_test,y_pred, average='weighted')

    rf_metrics = [accuracy_multi_log, recall_multi_log, f1_multi_log]

    cm = confusion_matrix(y_test, y_pred)
    cm_matrix = pd.DataFrame(data=cm, columns=['Actual 0', 'Actual 8', 'Actual 108'],
                             index=['Predict 0', 'Predict 8', 'Predict 108'])
    sns.heatmap(cm_matrix, annot=True, fmt='d', cmap='YlGnBu')

    plt.savefig('static/images/heatmap_rf.png')
    return rf_metrics

def MLP(hyperparams, X, y):

    clf = MLPClassifier(
        hidden_layer_sizes = hiperparams[0],
        activation = hiperparams[1],
        solver = hiperparams[2],
        alpha = hiperparams[3],
        random_state = hiperparams[4],
        max_iter = hiperparams[5]
    )

    clf = clf.fit(X,y)
    return clf
```

```
def MLP_class(model, X_test, y_test):

    #global multi_log_metrics

    y_pred = model.predict(X_test)

    accuracy_multi_log = accuracy_score(y_test, y_pred)
    recall_multi_log = recall_score(y_test, y_pred, average = None)
    f1_multi_log = f1_score(y_test,y_pred, average='weighted')

    mlp_metrics = [accuracy_multi_log, recall_multi_log, f1_multi_log]

    cm = confusion_matrix(y_test, y_pred)
    cm_matrix = pd.DataFrame(data=cm, columns=['Actual 0', 'Actual 8', 'Actual 108'],
                             index=['Predict 0', 'Predict 8', 'Predict 108'])
    sns.heatmap(cm_matrix, annot=True, fmt='d', cmap='YlGnBu')

    plt.savefig('static/images/heatmap_mlp.png')
    return mlp_metrics


def knn(hyperparams, X, y):

    neigh = KNeighborsClassifier(n_neighbors = hyperparams[0],
                                weights = hyperparams[1],
                                algorithm = hyperparams[2],
                                leaf_size = hyperparams[3])

    model = neigh.fit(X, y)

    return model


def knn_class(model, X_test, y_test):

    #global multi_log_metrics

    y_pred = model.predict(X_test)

    accuracy_multi_log = accuracy_score(y_test, y_pred)
    recall_multi_log = recall_score(y_test, y_pred, average = None)
    f1_multi_log = f1_score(y_test,y_pred, average='weighted')
```

```

knn_metrics = [accuracy_multi_log, recall_multi_log, f1_multi_log]

cm = confusion_matrix(y_test, y_pred)
cm_matrix = pd.DataFrame(data=cm, columns=['Actual 0', 'Actual 8', 'Actual 108'],
                        index=['Predict 0', 'Predict 8', 'Predict 108'])
sns.heatmap(cm_matrix, annot=True, fmt='d', cmap='YlGnBu')

plt.savefig('static/images/heatmap_knn.png')

return knn_metrics

```

A.2 Rotinas para ler a base de dados e concatenar Arquivos do mesmo evento:

```

def read_3w (folder_path, tipo_de_falha):
    global dados_final
    #tipo_de_falha = '2'
    lista = [i for i in range(9)]
    dados_final = []
    for pasta_atual, subpastas, arquivos in os.walk(folder_path):
        print(f'Diretorio atual: {pasta_atual}')
        print(f'Subdiretorios: {subpastas}')
        print(f'Arquivos: {arquivos}')
        n_pastas = len(subpastas)

        for pasta in subpastas:

            for item in lista:

                if str(pasta) == str(item):

                    if str(pasta) == str(tipo_de_falha):
                        #print(pasta, item, 'ok')
                        new_path = folder_path + '/' + pasta

                        for pasta_atual, subpastas, arquivos in os.walk(new_path):

                            dict_final = {}

```

```
        print('Lendo os arquivos do dataset ' + pasta)

        for file_final in arquivos:

            df_temporario = pl.read_csv(new_path + '/' +
                                         file_final,
                                         low_memory
                                         = False)

            dict_final[file_final] = df_temporario
            print(file_final)

            dados_final.append(dict_final)

        else: continue

    print("Ingestao de arquivos finalizada")
    return dados_final

def concatena_dataset():
    print(type(dados_final))
    keys = list(dados_final[0].keys())

    dataset_concatenado = pd.DataFrame()

    for chave in keys:

        temp = dados_final[0][chave]
        print(chave)
        df = pd.DataFrame(data = temp, columns= temp.columns)

        for coluna in list(df.columns):

            if coluna == 'timestamp':

                continue

            elif coluna == 'class':
                #df[coluna] = df[coluna].astype('int')
```

```
        continue

    else:

        df[coluna] = df[coluna].astype('float64')

    df = df.dropna(axis = 'columns', how='all')

    dataset_concatenado = pd.concat([dataset_concatenado,df])

dataset_concatenado = dataset_concatenado.dropna(axis=0, how='any')
return dataset_concatenado
```

Apêndice B

Segundo Apêndice

B.1 Arquivo principal para aplicacao DFPSE

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import polars as pl
import os

from fnmatch import fnmatch
import tkinter as tk
from tkinter import filedialog
import sklearn as skl

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
import time

from main_dfpse import *
```



```
app = Flask(__name__)
app.secret_key = '202419'

@app.route("/")
def home():
    return render_template("frontpage.html")

@app.route("/login", methods = ["POST", "GET"])
def login():
    if request.method == "POST":
        user = request.form["nm"]
        return redirect(url_for("user", usr=user))
    else:
        return render_template("login.html")

@app.route("/escolhe_dataset", methods = ["POST", "GET"])
def escolhe_dataset():
    if request.method == "POST":
        tipo_de_falha = str(request.form['dataset'])
        return redirect(url_for("resultados_modelos", tp_falha = tipo_de_falha))
    else:
        return render_template("escolhe_dataset.html")

@app.route("/resultados_evento_<tp_falha>/", methods = ["GET", "POST"])
def resultados(tp_falha):

    path_os = os.getcwd()
    path = f'{path_os}\\dataset'

    path_images = 'static\\images'

    flash("A funcao de leitura dos dados foi bem sucedida!")

    dados = read_3w(path, tp_falha)

    dados_falha = concatena_dataset()

    # Converter o DataFrame para uma lista de dicionarios
    session['path'] = path
```

```
session['tipo_de_falha'] = tp_falha

graficos = plota_grafico_boxplot(dados_falha)

imagens = []
# if arquivo.endswith((''.jpg', '.jpeg', '.png', '.gif')):

for arquivo in os.listdir(path_images):
    imagens.append(f'images/{arquivo}')#(os.path.join('images', arquivo))

if not dados:
    flash("A funcao read_3w nao retornou resultados. Verifique o diretorio e os
          dados.")

return render_template("falha_escolhida.html", tp_falha = tp_falha, imagens =
                      imagens)

@app.route("/resultados_modelos", methods = ["POST", "GET"])
def resultados_modelos():
    multi_log_metrics = []
    rf_metrics         = []
    knn_metrics        = []
    mlp_metrics        = []
    svm_metrics        = []

    path = session.get('path', None)
    tp_falha = session.get('tipo_de_falha', None)
    path_images = 'static\images'

    if not tp_falha:
        flash("Nenhum tipo de falha selecionado, volte para o passo ESCOLHA O DATASET.
              ")
        return redirect(url_for("escolhe_dataset"))
    else:

        dados = read_3w(path, tp_falha)

        dados_falha = concatena_dataset()
```

```
plota_grafico_boxplot(dados_falha)

plota_grafico_hist(dados_falha)

imagens_boxplot = []
imagens_histograma = []

for arquivo in os.listdir(f'{path_images}/caixa'):
    imagens_boxplot.append(arquivo)#(os.path.join('images', arquivo))

for arquivo in os.listdir(f'{path_images}/histograma'):
    imagens_histograma.append(arquivo)#(os.path.join('images', arquivo))

session['imagens_boxplot'] = imagens_boxplot
session['imagens_histograma'] = imagens_histograma

seed = 123459

X = dados_falha.drop(['timestamp', 'class'], axis = 1) #--> jogar na funcao de
                                                    concatenar

y = dados_falha['class'].astype('int') #--> isso tambem

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
                                                    random_state = seed)

#Modelo de Regressao Logistica
multi_log_params = ['multinomial'
                    , 'newton-cg'
                    , 10000
                    ]
regressao = multi_log_regress(multi_log_params, X_train, y_train)

multi_log_metrics = multi_log_class(regressao, X_test, y_test)

# Fim da regressao logistica
```

```
#Modelo de Random Forest

rf_params = ['gini',
             ,True
             ]

rf = random_forest(rf_params, X_train, y_train)

rf_metrics = random_forest_class(rf, X_test, y_test)

# Fim da Random Forest

#Modelo de KNN

knn_params = [5,
              , 'uniform',
              , 'auto',
              40]

knn_model = knn(knn_params, X_train, y_train)

knn_metrics = knn_class(knn_model, X_test, y_test)

# Fim da KNN

#Modelo de MLP

mlp_params = [100
              , 'relu'
              , 'adam'
              , 0.0001
              , None
              , 200]

mlp_model = MLP(mlp_params, X_train, y_train)

mlp_metrics = MLP_class(mlp_model, X_test, y_test)

# Fim da MLP

#Modelo de svm
```

```

svm_params = [10.0
               , 'rbf'
               , 0.3
               , 3]

svm_model = svm(svm_params, X_train, y_train)

svm_metrics = svm_class(svm_model, X_test, y_test)

# Fim da MLP

return render_template("resultados_modelos.html"
                       , tp_falha = tp_falha
                       , multi_log_params = multi_log_params
                       , metric_multi_log = multi_log_metrics
                       , rf_params = rf_params
                       , metric_rf = rf_metrics
                       , knn_params = knn_params
                       , metric_knn = knn_metrics
                       , mlp_params = mlp_params
                       , metrics_mlp = mlp_metrics
                       , svm_params = svm_params
                       , metrics_svm = svm_metrics)

@app.route("/analise-exploratoria", methods = ["POST", "GET"])
def analise_exploratoria():

    path = session.get('path', None)
    tp_falha = session.get('tipo_de_falha', None)
    imagens_boxplot = session.get('imagens_boxplot', None)
    imagens_histograma = session.get('imagens_histograma', None)

    if not tp_falha:
        flash("Nenhum tipo de falha selecionado, volte para o passo ESCOLHA O DATASET.
              ")

        return redirect(url_for("escolhe_dataset"))
    else:

```

```
    return render_template("falha_escolhida_2.html", tp_falha = tp_falha,
                           imagens_boxplot = imagens_boxplot,
                           imagens_historico =
                           imagens_historico )
```

B.2 Funcoes utilizadas dentro da Aplicacao DFPSE

```
#tratamento e concatenacao de dataset dependendo da pasta
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import polars as pl
import os
from fnmatch import fnmatch
import tkinter as tk
from tkinter import filedialog
import sklearn as skl
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
import time
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
import matplotlib
matplotlib.use('agg')

#tipo de falha -> Numero entre 0 a 8 (0 - normal, 1-8 - Falha)

def concatena_dataset():
    print(type(dados_final))
```

```
keys = list(dados_final[0].keys())

dataset_concatenado = pd.DataFrame()

for chave in keys:

    temp = dados_final[0][chave]
    print(chave)
    df = pd.DataFrame(data = temp, columns= temp.columns)

    for coluna in list(df.columns):

        if coluna == 'timestamp':

            continue

        elif coluna == 'class':
            #df[coluna] = df[coluna].astype('int')
            continue

        else:

            df[coluna] = df[coluna].astype('float64')

    df = df.dropna(axis = 'columns', how='all')

    dataset_concatenado = pd.concat([dataset_concatenado,df])

dataset_concatenado = dataset_concatenado.dropna(axis=0, how='any')
return dataset_concatenado

def plota_grafico_boxplot(df):

    df = df.drop(columns = ['timestamp', 'class' ])
    colunas = list(df.columns)

    # path_os = os.getcwd()
```

```
# pasta = rf'{path_os}\dataset\static\images\boxplot'

pasta = 'static\images'

for arquivo in os.listdir(pasta):
    caminho_arquivo = os.path.join(pasta, arquivo)
    if os.path.isfile(caminho_arquivo):
        os.remove(caminho_arquivo)

for i in range(1, len(colunas)):

    plt.figure(figsize=(20, 10))
    fig = df.boxplot(column=colunas[i])
    fig.set_title('')
    fig.set_ylabel(colunas[i])

    plt.savefig(f'{pasta}/grafico{i}.jpg')

#Hiperparams deve ser uma lista contendo:
# 0 -> multi_class : 'auto', 'ovr', 'multinomial'
# 1 -> solver: 'newton-cg', 'sag', 'saga' and 'lbfgs'
# 2 -> max_iter: Numero de iteracoes, default 10000

def multi_log_regress(hiperparams, X_treino, Y_treino):
    model = LogisticRegression(
        multi_class = hiperparams[0]
        , solver = hiperparams[1]
        , max_iter = hiperparams[2]
        , class_weight = None
    )

    model.fit(X_treino, Y_treino)

    return model

def multi_log_class(model, X_test, y_test):
    #global multi_log_metrics
    plt.clf()
```



```

y_pred = model.predict(X_test)

accuracy_multi_log = accuracy_score(y_test, y_pred)
recall_multi_log = recall_score(y_test, y_pred, average = None)
f1_multi_log = f1_score(y_test,y_pred, average='weighted')

multi_log_metrics = [accuracy_multi_log, recall_multi_log, f1_multi_log]

cm_log = confusion_matrix(y_test, y_pred)
cm_matrix_log = pd.DataFrame(data=cm_log, columns=['Actual 0', 'Actual 8', 'Actual
                                                108'],
                             index=['Predict 0', 'Predict 8', 'Predict 108'])
sns.heatmap(cm_matrix_log, annot=True, fmt='d', cmap='YlGnBu')

plt.savefig('static/images/heatmap_multilog.png')
return multi_log_metrics

def svm (hiperparams, X_treino, Y_treino):
    model = SVC(
        C = hiperparams[0] #10
        , kernel= hiperparams[1] #'rbg'
        , gamma = hiperparams[2] #0.3
        , degree= hiperparams[3] # 3
    )

    model.fit(X_treino, Y_treino)
    return model

def svm_class(model, X_test, y_test):
    #global multi_log_metrics
    plt.clf()
    y_pred = model.predict(X_test)

    accuracy_multi_log = accuracy_score(y_test, y_pred)
    recall_multi_log = recall_score(y_test, y_pred, average = None)
    f1_multi_log = f1_score(y_test,y_pred, average='weighted')

    multi_log_metrics = [accuracy_multi_log, recall_multi_log, f1_multi_log]

```

```
cm_svm = confusion_matrix(y_test, y_pred)
cm_matrix_svm = pd.DataFrame(data=cm_svm, columns=['Actual 0', 'Actual 8', 'Actual
                                                108'],
                             index=['Predict 0', 'Predict 8', 'Predict 108'])
sns.heatmap(cm_matrix_svm, annot=True, fmt='d', cmap='YlGnBu')

plt.savefig('static/images/heatmap_svm.png')
return multi_log_metrics

def random_forest(hiperparams, X_treino, Y_treino):
    model = RandomForestClassifier(
        criterion = hiperparams[0]
        , oob_score = hiperparams[1]
    )

    model.fit(X_treino, Y_treino)
    return model

def random_forest_class(model, X_test, y_test):
    #global multi_log_metrics
    plt.clf()
    y_pred = model.predict(X_test)

    accuracy_multi_log = accuracy_score(y_test, y_pred)
    recall_multi_log = recall_score(y_test, y_pred, average = None)
    f1_multi_log = f1_score(y_test,y_pred, average='weighted')

    rf_metrics = [accuracy_multi_log, recall_multi_log, f1_multi_log]

    cm_rf = confusion_matrix(y_test, y_pred)
    cm_matrix_rf = pd.DataFrame(data=cm_rf, columns=['Actual 0', 'Actual 8', 'Actual
                                                108'],
                             index=['Predict 0', 'Predict 8', 'Predict 108'])
    sns.heatmap(cm_matrix_rf, annot=True, fmt='d', cmap='YlGnBu')

    plt.savefig('static/images/heatmap_rf.png')
    return rf_metrics
```

```
def MLP(hyperparams, X, y):
```

```
    clf = MLPClassifier(
        hidden_layer_sizes = hyperparams[0],
        activation = hyperparams[1],
        solver = hyperparams[2],
        alpha = hyperparams[3],
        random_state = hyperparams[4],
        max_iter = hyperparams[5]
    )
```

```
    clf = clf.fit(X,y)
    return clf
```

```
def MLP_class(model, X_test, y_test):
```

```
    #global multi_log_metrics
```

```
    plt.clf()
```

```
    y_pred = model.predict(X_test)
```

```
    accuracy_multi_log = accuracy_score(y_test, y_pred)
```

```
    recall_multi_log = recall_score(y_test, y_pred, average = None)
```

```
    f1_multi_log = f1_score(y_test,y_pred, average='weighted')
```

```
    mlp_metrics = [accuracy_multi_log, recall_multi_log, f1_multi_log]
```

```
    cm_mlp = confusion_matrix(y_test, y_pred)
```

```
    cm_matrix_mlp = pd.DataFrame(data=cm_mlp, columns=['Actual 0', 'Actual 8', 'Actual
                                                    108'],
                                index=['Predict 0', 'Predict 8', 'Predict 108'])
```

```
    sns.heatmap(cm_matrix_mlp, annot=True, fmt='d', cmap='YlGnBu')
```

```
    plt.savefig('static/images/heatmap_mlp.png')
```

```
    return mlp_metrics
```

```
def knn(hyperparams, X, y):
```

```
    neigh = KNeighborsClassifier(n_neighbors = hyperparams[0],
```

```

        weights = hyperparams[1],
        algorithm = hyperparams[2],
        leaf_size = hyperparams[3])

model = neigh.fit(X, y)

return model

def knn_class(model, X_test, y_test):
    #global multi_log_metrics
    plt.clf()
    y_pred = model.predict(X_test)

    accuracy_multi_log = accuracy_score(y_test, y_pred)
    recall_multi_log = recall_score(y_test, y_pred, average = None)
    f1_multi_log = f1_score(y_test, y_pred, average='weighted')

    knn_metrics = [accuracy_multi_log, recall_multi_log, f1_multi_log]

    cm_knn = confusion_matrix(y_test, y_pred)
    cm_matrix_knn = pd.DataFrame(data=cm_knn, columns=['Actual 0', 'Actual 8', 'Actual
                                                    108'],
                                index=['Predict 0', 'Predict 8', 'Predict 108'])
    sns.heatmap(cm_matrix_knn, annot=True, fmt='d', cmap='YlGnBu')

    plt.savefig('static/images/heatmap_knn.png')
    return knn_metrics

def read_3w (folder_path, tipo_de_falha):
    global dados_final
    #tipo_de_falha = '2'
    lista = [i for i in range(9)]
    dados_final = []
    for pasta_atual, subpastas, arquivos in os.walk(folder_path):
        print(f'Diretorio atual: {pasta_atual}')
        print(f'Subdiretorios: {subpastas}')
        print(f'Arquivos: {arquivos}')
        n_pastas = len(subpastas)

        for pasta in subpastas:

```

```
for item in lista:

    if str(pasta) == str(item):

        if str(pasta) == str(tipo_de_falha):
            #print(pasta, item, 'ok')
            new_path = folder_path + '/' + pasta

            for pasta_atual, subpastas, arquivos in os.walk(new_path):

                dict_final = {}
                print('Lendo os arquivos do dataset ' + pasta)

                for file_final in arquivos:

                    df_temporario = pl.read_csv(new_path + '/' +
                                                    file_final,
                                                    low_memory
                                                    = False)

                    dict_final[file_final] = df_temporario
                    print(file_final)

                dados_final.append(dict_final)

            else: continue

print("Ingestao de arquivos finalizada")
return dados_final
```